

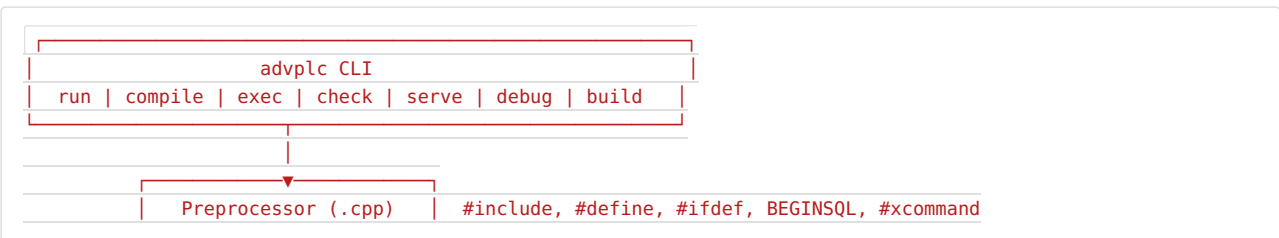
Guia do Desenvolvedor — AdvPP Compiler

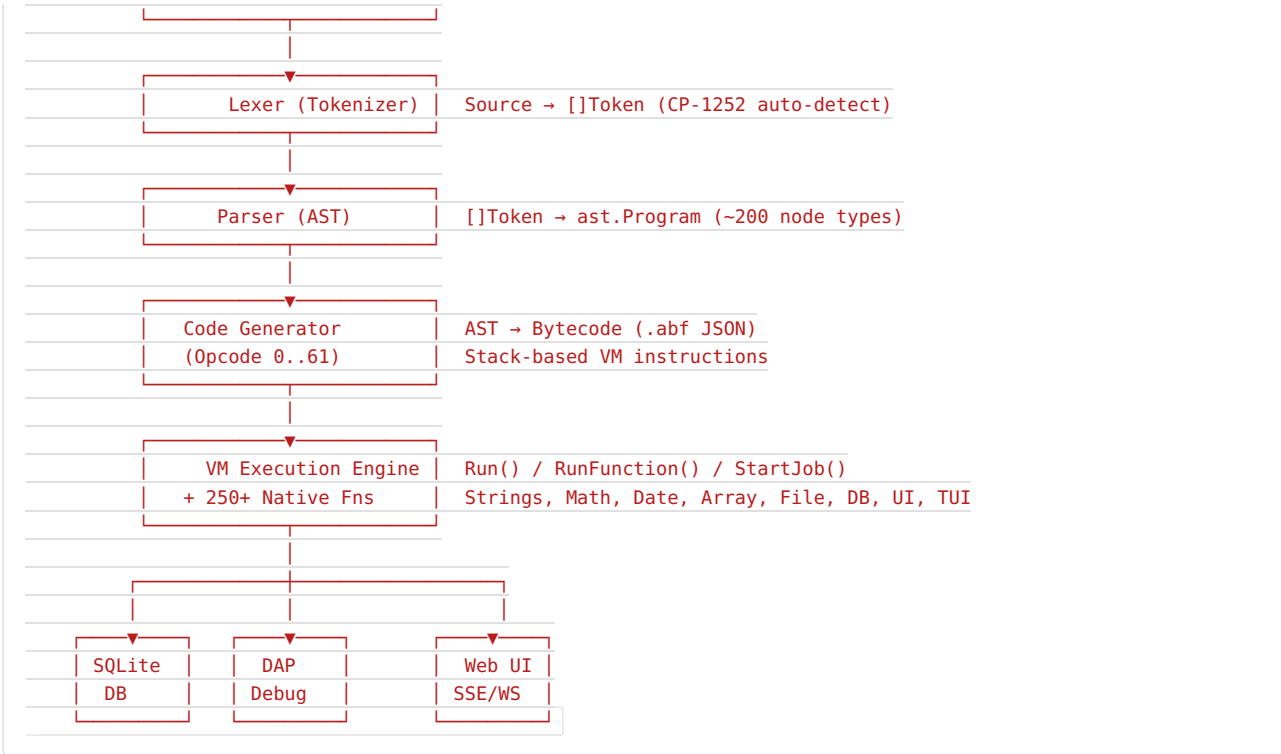
Compilador AdvPL/TLPP em Go · github.com/advpl/compiler · 137 arquivos .go (não-teste) · ~48.490 linhas

Índice

- [Visão Geral da Arquitetura](#)
- [CLI — advplc](#)
- [Pipeline de Compilação](#)
 - [3.1 Conversão de Codificação \(CP-1252 → UTF-8\)](#)
 - [3.2 Preprocessor](#)
 - [3.3 Lexer](#)
 - [3.4 Parser](#)
 - [3.5 Code Generation / Bytecode](#)
- [Máquina Virtual \(VM\)](#)
 - [4.1 Tipos de Valor \(`pkg/runtime/values.go` \)](#)
 - [4.2 Ambiente de Variáveis \(`pkg/runtime/environment.go` \)](#)
 - [4.3 VM Core \(`pkg/vm/vm.go` \)](#)
 - [4.4 Funções Nativas Registradas \(~250+\)](#)
 - [4.4.1 TUI / Terminal \(`pkg/vm/ui\_render.go` \)](#)
 - [4.5 Sistema de Diálogo \(MsDialog\)](#)
 - [4.6 Browse \(FWMBrowse Auto-CRUD\)](#)
 - [4.7 Grid Process \(FWGridProcess\)](#)
 - [4.8 Macro Substitution \(&\)](#)
 - [4.9 Debugger](#)
 - [4.10 Tensor \(Operações com Tensores\)](#)
 - [4.11 Autograd \(Diferenciação Automática\)](#)
 - [4.12 LLM \(Inferência GGUF\)](#)
 - [4.13 TMailMessage \(E-mail SMTP\)](#)
 - [4.14 MCPServer \(Model Context Protocol\)](#)
 - [4.15 WSRestServer \(REST API\)](#)
- [Framework MVC](#)
- [UI Desktop \(Fyne\)](#)
- [UI Web \(PO-UI\)](#)
- [Banco de Dados SQLite](#)
- [Protocolos \(DAP, MCP, REST\)](#)
- [Build Standalone](#)
- [1. Tabela de Opcodes do Bytecode](#)
- [2. Resumo de Coverage](#)

1. Visão Geral da Arquitetura





Estrutura de Pacotes

Pacote	Arquivos	Responsabilidade
<code>cmd/advplc</code>	main.go (+17 testes)	CLI principal: compile, run, serve, debug, build, check
<code>cmd/advpp-ide</code>	main.go	IDE desktop completo com editor Fyne
<code>cmd/adveditor</code>	main.go	Editor de código com banco SQLite
<code>pkg/compiler</code>	6 arquivos	Bytecode compiler, opcodes, standalone builder
<code>pkg/lexer</code>	2 arquivos	Tokenizer do idioma AdvPL/TLPP
<code>pkg/parser</code>	2 arquivos	Parser recursivo descendente, expressions
<code>pkg/ast</code>	1 arquivo	~200 tipos de nós AST
<code>pkg/preprocessor</code>	2 arquivos	<code>#include</code> , <code>#define</code> , <code>#ifdef</code> , <code>#xcommand</code> , SQL blocks
<code>pkg/runtime</code>	3 arquivos	Tipos de valor dinâmico, ambiente de escopo
<code>pkg/vm</code>	15 arquivos	VM execution engine + todas as nativas
<code>pkg/mvc</code>	4 arquivos	FWFormModel, FWFormView, FWFormBrowse
<code>pkg/ui</code>	11 arquivos	Widgets Fyne: editor, console, file tree, renderer MVC
<code>pkg/webui</code>	1 arquivo	Servidor HTTP com SSE para modo web PO-UI
<code>pkg/db</code>	1 arquivo	Motor SQLite com workarea Protheus
<code>pkg/dap</code>	3 arquivos	Debug Adapter Protocol
<code>pkg/mcp</code>	2 arquivos	Model Context Protocol JSON-RPC
<code>pkg/rest</code>	1 arquivo	Servidor REST HTTP

<code>pkg/tensor</code>	4 arquivos	Tensor N-dimensional, linear algebra
<code>pkg/autograd</code>	5 arquivos	Reverse-mode autodiff
<code>pkg/nn</code>	2 arquivos	Camadas neurais (Linear, Embedding)
<code>pkg/llm</code>	11 arquivos	Inferência GGUF I2_S (BitNet), tokenizers
<code>pkg/tools/shared</code>	5 arquivos	Configuração compartilhada, database, treeview

2. CLI — advplc

Comandos

Comando	Descrição	Exemplo
<code>run</code>	Compila e executa um fonte AdvPL	<code>advplc run programa.prw</code>
<code>compile</code>	Compila para bytecode (.abf JSON)	<code>advplc compile programa.prw -o out.abf</code>
<code>exec</code>	Executa bytecode já compilado	<code>advplc exec out.abf</code>
<code>check</code>	Verifica sintaxe sem executar (multi-arquivo)	<code>advplc check a.prw b.prw c.prw</code>
<code>serve</code>	Modo web — UI no browser via PO-UI/Angular	<code>advplc serve programa.prw --port 9000</code>
<code>debug</code>	DAP server para IDE debugging	<code>advplc debug</code> (pipe stdin/stdout do editor)
<code>build</code>	Build executável standalone	<code>advplc build programa.prw -o Programa</code>
<code>ast</code>	Imprime a estrutura AST	<code>advplc ast programa.prw</code>
<code>bytecode</code>	Imprime bytecode instructions	<code>advplc bytecode programa.prw</code>
<code>version</code> / <code>-v</code>	Versão do compilador	<code>advplc -v</code>

Opções Globais

Opção	Descrição
<code>--include <path></code> , <code>-I <path></code>	Adiciona caminho para busca de includes (repetível)
<code>--define <nome=val></code> , <code>-D</code>	Define símbolo no pré-processor
<code>--ui</code>	Habilita interface gráfica (Fyne desktop)
<code>--headless</code>	Desabilita interface (padrão)
<code>--db <backend></code>	Aceito mas sem efeito hoje — só <code>sqlite</code> está implementado
<code>--db-path <caminho></code>	Path do arquivo SQLite
<code>--port</code>	

<code><n></code>	Porta do servidor web (modo serve)
<code>--watch</code> , <code>-w</code>	Hot reload: recompila ao detectar mudança no fonte
<code>--debug-port <n></code>	Abre listener TCP DAP para attach debugging
<code>-o</code> <code><arquivo></code>	Arquivo de saída (para <code>compile</code> e <code>build</code>)
<code>--gui</code>	<code>build</code> : fixa o binário como app desktop — janela Fyne sempre, sem tentar console; no Windows linka no subsistema GUI (<code>-H=windowsgui</code> , ldflags, ver <code>goBuildArgs</code> em <code>pkg/compiler/standalone.go</code>), sem flash de console. Obrigatório para programas com <code>MSDIALOG</code> . Sem essa flag, o binário standalone decide sozinho em runtime (console se TTY + nada de UI detectado no bytecode; Fyne caso contrário) — ver <code>pkg/compiler/stub_template.go</code> . <code>ADVPP_FORCE_GUI=1</code> é o equivalente em runtime, mais fraco (não muda o subsistema Windows).

Pipeline Interno de `LoadAndCompile()`

1. `os.ReadFile(sourceFile)` — lê bytes brutos
2. `convertToUTF8()` — detecta CP-1252 e converte para UTF-8 (tabela lookup 256 bytes)
3. `preprocessor.Process()` — processa `#include`, `#define`, `#ifdef`, `BEGINSQL`, `#xcommand`
4. `lexer.Tokenize()` — gera `[]Token` da fonte pré-processada
5. `parser.Parse()` — gera `*ast.Program`
6. `compiler.Compile()` — gera `*compiler.Bytecode`

3. Pipeline de Compilação

3.1 Conversão de Codificação (CP-1252 → UTF-8)

Função: `convertToUTF8(source []byte) (string, error)` em `cmd/advplc/main.go`

Detecta se o source é UTF-8 válido. Se não, assume CP-1252 e converte usando uma tabela lookup de 256 entradas construída em `init()`. Blocos ASCII contíguos são copiados de uma vez (`buf.Write(source[start:i])`), maximizando performance.

Cobertura CP-1252: Mapeia todos os 256 valores de byte incluindo caracteres especiais em 0x80–0x9F (€, ,, ..., †, ‡, ^, %, Š, ‹, Œ, Ž, " "", •, ¬, —, ~, ™, etc.)

3.2 Preprocessador (`pkg/preprocessor/`)

Tipo Principal: `Preprocessor`

Método	Descrição
<code>NewPreprocessor(includePaths []string)</code>	Cria preprocessador com caminhos de busca para <code>#include</code>
<code>Process(source, fileName string) (string, error)</code>	Aplica todos os preprocessamentos, retorna texto transformado
<code>GetDefines() map[string]string</code>	Retorna defines ativos (<code>#define</code>)
<code>RootBoundaryLine int</code>	Linha limite — includes aparecem antes desta linha

Diretivas Suportadas

Diretiva	Comportamento

<code>#include <file></code>	Carrega recursivamente (depth max 30); inclui antes do RootBoundaryLine
<code>#define nome valor</code>	Armazena no mapa defines; suporta continuação com <code>;</code>
<code>#undef</code> / <code>#undef nome</code>	Remove define
<code>#ifdef nome</code> / <code>#ifndef nome</code>	Condição de compilação condicional (aninhável)
<code>#else</code> / <code>#endif</code>	Controle de ramos condicionais
<code>#xcommand ... => ...</code> / <code>#command</code>	Macro DSL: pattern → result template (recompilação automática de código)
<code>#xtranslate ... => ...</code> / <code>#translate</code>	Similar a <code>#command</code> para tradução de tokens
<code>BEGINSQL [ALIAS nome]</code>	Coleta SQL até <code>ENDSQL</code> ; converte em string-building AdvPL
Continuação de linha (<code>;</code> no final)	Junta próxima linha física

Command Rules Engine (#xcommand)

Compile padrões DSL em regras internas (`commandRule`) que são aplicadas a cada linha de código. Pipeline:

- 1. `parseCommandDef(def)` — separa `pattern => result`
- 2. `compilePattern(s)` — cria tokens: literais, marcadores `<name>` , opcionais `[...]` , restritos `<name:LIT1,LIT2>`
- 3. `matchPattern(pattern, tokens)` — backtracking recursive matcher
- 4. `expandResult(result, match)` — substitui marcadores por texto capturado: `<name>` = raw text, `<{name}>` = codeblock `{|| text}` , `<.name.>` = boolean `.T./.F.`

3.3 Lexer (`pkg/lexer/`)

Tipo Principal: `Lexer`

Função	Parâmetros	Retorno	Descrição
<code>Tokenize(source, fileName string)</code> <code>([]Token, error)</code>	fonte + nome arquivo	tokens	Cria lexer e tokeniza em uma chamada
<code>FilterTokens(tokens []Token) ([]Token, error)</code>	tokens brutos	tokens limpos	Remove <code>TOKEN_NEWLINE</code> , <code>LINECOMMENT</code> , <code>BLOCKCOMMENT</code>

Token Types (~43 tipos)

Literais:	<code>TOKEN_NUMBER</code> , <code>TOKEN_STRING</code> , <code>TOKEN_IDENT</code>
Lógicos:	<code>TOKEN_TRUE (.T./.Y.)</code> , <code>TOKEN_FALSE (.F./.N.)</code> , <code>TOKEN_NIL (.NULL./.NIL.)</code>
Aritméticos:	<code>TOKEN_PLUS(+)</code> , <code>TOKEN_MINUS(-)</code> , <code>TOKEN_STAR(*)</code> , <code>TOKEN_SLASH(/)</code> , <code>TOKEN_PERCENT(%)</code>
Comparação:	<code>TOKEN_EQ(==)</code> , <code>TOKEN_NEQ(!=</>#)</code> , <code>TOKEN_LT(<)</code> , <code>TOKEN_GT(>)</code> , <code>TOKEN_LTE(<=)</code> , <code>TOKEN_GTE(>=)</code>
Lógicos prefix:	<code>TOKEN_DOT_AND(.AND.)</code> , <code>TOKEN_DOT_OR(.OR.)</code> , <code>TOKEN_DOT_NOT(.NOT.)</code>
Incremento:	<code>TOKEN_INCREMENT(++)</code> , <code>TOKEN_DECREMENT(--)</code>
Potência:	<code>TOKEN_CARET(^)</code> , <code>TOKEN_DOUBLESTAR(**)</code>
Ponctuação:	<code>LPAREN</code> , <code>RPAREN</code> , <code>LBRACKET</code> , <code>RBRACKET</code> , <code>LBRACE</code> , <code>RBRACE</code> , <code>SEMICOLON</code> , <code>COMMA</code> , <code>DOT(.)</code> , <code>COLON(:)</code> , <code>DOUBLE</code>
Avanço:	<code>AT(@)</code> , <code>AMPERSAND(&)</code> , <code>PIPE()</code> , <code>TILDE(~)</code> , <code>DOLLAR(\$)</code>
Pré-processador:	<code>TOKEN_PREPROC_INCLUDE</code> , <code>DEFINE</code> , <code>UNDEFINE</code> , <code>IFDEF</code> , <code>IFNDEF</code> , <code>ELSE</code> , <code>ENDIF</code> , <code>XCOMMAND</code> , <code>XTRANSLATE</code> , <code>CON</code>
Comentários:	<code>TOKEN_LINECOMMENT(//,&&)</code> , <code>TOKEN_BLOCKCOMMENT(/* */)</code>

Keywords (80+ palavras reservadas)

`FUNCTION` , `STATIC` , `USER` , `PROCEDURE` , `RETURN` , `MAIN` , `IF` , `ELSEIF` , `ELSE` , `ENDIF` , `FOR` , `TO` , `STEP` , `NEXT` , `WHILE` , `ENDDO` , `END` , `DO` , `CASE` , `ENDCASE` , `OTHERWISE` , `EXIT` , `LOOP` , `BREAK` , `CONTINUE` , `BEGIN` , `SEQUENCE` , `RECOVER` , `USING` , `TRY` , `CATCH` , `FINALLY` , `ENDTRY` , `THROW` , `LOCAL` , `PRIVATE` , `PUBLIC` , `GLOBAL` , `PARAMETERS` , `CLASS` , `ENDCLASS` , `DATA` , `METHOD` , `CONSTRUCTOR` , `FROM` , `AS` , `OPERATOR` , `INTERFACE` , `ENDINTERFACE` , `IMPLEMENTS` , `VARIANT` , `VARIADIC` , `JSON` , `INTEGER` , `DOUBLE` , `DECIMAL` , `CHARACTER` , `NUMERIC` , `LOGICAL` , `DATE` , `ARRAY` , `OBJECT` , `MEMO` , `FIXED` , `UNDEFINED` , `AND` , `OR` , `NOT` , `IN` , `IS` , `OF` , `ON` , `THREAD` , `JOB` , `EXPORT` , `ENUM` , `ENDENUM` , `REST` , `GET` , `POST` , `PUT` , `DELETE` , `PATCH` ,

NAMESPACE, SAY, SIZE, PIXEL, COORD, TITLE, ACTION, WHEN, VALID, UPDATE, READ, ADD, ACTIVATE, WINDOW, DIALOG, BUTTON, SBUTTON, MENU, BROWSE, REPORT, SECTION, PANEL, BLOCK, INCLUDE, TRANSLATE, COMMAND, UNDEFINE, IFDEF, IFNDEF

Comportamentos Especiais do Lexer

- NBSP (0xC2 0xA0): Detecta UTF-8 encode de non-breaking space
- Bracket strings: [texto] funciona como literal string ou array index dependendo do contexto
- Dot literals: .T., .F., .Y., .N., .NULL., .AND., .OR., .NOT. são tokens unitários
- Continuação de linha: ; no final da linha une com próxima linha física

3.4 Parser (pkg/parser/)

Tipo Principal: Parser

Método	Parâmetros	Retorno	Descrição
NewParser(tokens, fileName, defines)	tokens lexados	*Parser	Cria parser com tokens
Parse() (*ast.Program, error)	—	AST root	Parseia stream de tokens em programa completo

Estratégia de Parsing

- Statements: Pattern keyword-dispatch — peeks primeiro token, matching keyword, dispatch handler específico
- Expressions: Pratt top-down operator precedence chain

Precência de Operadores (lowest → highest)

```
parseExpression
→ parseOr (.Or./OR)
→ parseAnd (.And./AND)
→ parseNot (.Not./!)
→ parseComparison (==, =, !=, <=>, <=, >=, $)
→ parseAddition (+, -)
→ parseMultiplication (*, /, %)
→ parsePower (^, **) ← right associative
→ parseUnary (-, +, ++, --)
→ parsePostfix (:method(), [idx], ->field, (), ++, -- suffix)
→ parsePrimary (literals, identifiers, macros, JSON)
```

DSL Commands Desugaring

Todos os commands legacy AdvPL (DEFINE/REDEFINE, @SCREEN SAY/GET, CLIPPER DB commands) são parsados e convertidos em CallExpr — preservam a estrutura syntax sem requerer runtime engine dedicado. São "degenerated" em chamadas de função nomeadas como DESLUGGED_DEFINE_MS_DIALOG , etc.

3.5 Code Generation / Bytecode (pkg/compiler/)

Tipo Principal: Compiler

Método	Parâmetros	Retorno	Descrição
New()	—	*Compiler	Cria novo compiler com bytecode vazio
Compile(program *ast.Program)	AST tree	*Bytecode, error	Entry point — compila todo AST para bytecode

Otimizações do Codegen

- Constant pool deduplication: números e strings repetidos compartilham referência
- Inline IF()/IIFO): special case — short-circuit evaluation (só avalia ramo escolhido)
- EVAL(codeblock): usa OP_EVAL_CODEBLOCK ao invés de OP_CALL_NATIVE
- Closure upvalue capture: resolve variáveis de frames parentais para closures
- Global bit flag: locals globais são marcados com bit 0x8000 no slot index

- **Reserved function names:** builtins (ERRORCLASS, JSONOBJECT, FWFORMVIEW, etc.) vão para `OP_NEW_INSTANCE`
- **Named parameters:** cada argumento tem marker `OP_NAMED_ARG` antes do valor real

Classes Built-in Geradas Automaticamente

`ERRORCLASS`, `JSONOBJECT`, `JSONARRAY`, `FWFORMVIEW`, `FWFORMMODEL`, `FWFORMBROWSE`, `FWGRIDPROCESS`, `FWMBROWSE`, `LLM`, `MCPSERVER`, `WSRESTERSERVER`, `TMAILMESSAGE`, `TENSOR`, `VARIABLE`, `SGD`, `ADAM`, `LINEAR`, `EMBEDDING`

Serialização de Bytecode

Função	Descrição
<code>SaveBytecode(bc, filename string)</code>	Marshal → JSON file
<code>LoadBytecode(filename string)</code>	Unmarshal JSON → <code>*Bytecode</code>
<code>BuildStandaloneGUI(bc, outputFile, title, gui, logWriter)</code>	Embed bytecode em stub Go compilável → executável native (<code>gui bool</code> = flag <code>--gui</code> , ver seção 10)

4. Máquina Virtual (VM)

4.1 Tipos de Valor (`pkg/runtime/values.go`)

Interface base: `Value`

Método	Retorno	Descrição
<code>Type()</code>	<code>string</code>	Character de tipo: "N"/"C"/"L"/"D"/"A"/"B"/"O"/"U"/"E"
<code>String()</code>	<code>string</code>	Representação textual
<code>IsTruthy()</code>	<code>bool</code>	Truthiness: nil/empty = false
<code>Equals(other Value)</code>	<code>bool</code>	Igualdade semântica
<code>IsNil()</code>	<code>bool</code>	É NIL?

Tipos Implementados

Tipo	Struct	Field	Description
Undefined/Nil	<code>NilValue</code>	(singleton)	AdvPL <code>NIL</code> — <code>Type()</code> = "U"
Number	<code>NumberValue</code>	<code>Val float64</code>	Caching de inteiros -256..4096
String	<code>StringValue</code>	<code>Val string</code>	Strings UTF-8
Logical	<code>BoolValue</code>	<code>Val bool</code>	Singleton True/False
Date	<code>DateValue</code>	<code>Val time.Time</code>	Dates como <code>time.Time</code> Go
Array	<code>ArrayValue</code>	<code>Elements []Value</code>	Arrays multidimensionais
Code Block	<code>CodeBlockValue</code>	<code>Params, Body, Env, FuncName, Upvalues</code>	Closures com upvalues
Object	<code>ObjectValue</code>	<code>ClassName, Props map[string]Value, Keys, Class, Native</code>	JsonObject e objetos framework
Function	<code>FunctionValue</code>	<code>Name, Fn func([]Value)(Value, error)</code>	Native functions da VM

Error	ErrorValue	Description, Severity, Stack, ClassName, GenCode	Catchable via Try/Catch
-------	------------	--	-------------------------

Funções Fábrica

Função	Parâmetros	Retorno
Nil()	—	*NilValue
True() / False()	—	*BoolValue
NewNumber(v float64)	valor numérico	*NumberValue
NewString(s string)	texto	*StringValue
NewBool(b bool)	booleano	*BoolValue
NewDate(t time.Time)	data/hora	*DateValue
NewArray(elems []Value)	elementos	*ArrayValue
NewError(desc string)	descrição	*ErrorValue
ToFloat(v Value)	qualquer valor	float64
ToString(v Value)	qualquer valor	string
ToBool(v Value)	qualquer valor	bool
ValType(v Value)	qualquer valor	string ("N", "C", "L", "A", "B", "O", "U", "D")

4.2 Ambiente de Variáveis (pkg/runtime/environment.go)

Implementa variáveis Private/Public com scoped chaining.

Método	Descrição
NewEnvironment(parent)	Cria scope vinculado a parent
Define(name, value)	Define variável no scope atual (uppercase normalizado)
Get(name)	Lookup na cadeia (scope atual → pais → avô...)
Set(name, value)	Set no scope atual ou ancestral
Has(name)	Verifica existência

4.3 VM Core (pkg/vm/vm.go)

Tipos Principais

DBEngine interface — abstração de workarea Protheus

Método	Retorno	Descrição
SelectArea(alias string)	error	Seleciona área de trabalho (tabela)
Seek(key string)	(bool, error)	Busca chave na ordem atual
Skip(count int)	error	Move N registros
GoTop()	error	Posiciona no primeiro
GoBottom()	error	Posiciona no último

<code>EOF()</code>	bool	No fim do arquivo?
<code>BOF()</code>	bool	Antes do início?
<code>FieldGet(field string)</code>	(Value, error)	Lê campo
<code>FieldPut(field string, val Value)</code>	error	Escreve campo
<code>RecLock()</code>	error	Bloqueia registro
<code>MsUnlock()</code>	error	Desbloqueia e salva
<code>Append()</code>	error	Insere novo registro
<code>RecCount()</code>	int	Total de registros
<code>RecNo()</code>	int	Número do registro atual (1-based)
<code>FieldPos(field string)</code>	int	Posição do campo (1-based)

UIProvider interface — abstração de interação com usuário

Método	Retorno	Descrição
<code>MsgInfo(msg, title string)</code>	—	Caixa de informação
<code>MsgStop(msg, title string)</code>	—	Caixa de erro
<code>MsgAlert(msg, title string)</code>	—	Caixa de alerta
<code>MsgYesNo(msg, title string)</code>	bool	Confirmação sim/não
<code>Menu(items []string, title string)</code>	int	Menu seleção (1-based, 0=cancelado)
<code>InputText(prompt, def string, bIsPassword bool)</code>	string	Input de texto (maskable)
<code>Browse(spec []byte)</code>	<code>[]byte</code>	Renderiza FWMBrowse (web/desktop)
<code>Dialog(spec []byte)</code>	<code>[]byte</code>	Renderiza MSDIALOG (web/desktop)

SignalKind enum: `SigNone`, `SigReturn`, `SigExit`, `SigLoop`, `SigBreak`

Métodos da VM

Método	Parâmetros	Retorno	Descrição
<code>NewVM(bc, uiEnabled)</code>	bytecode, flag UI	<code>*VM</code>	Factory: registra classes + nativas, cria stack vazia
<code>SetDBEngine(engine)</code>	<code>DBEngine</code>	—	Define engine de banco
<code>SetDBFactory(factory)</code>	<code>func()</code> <code>DBEngine</code>	—	Factory per-job (StartJob abre conexão própria)
<code>SetUIProvider(provider)</code>	<code>UIProvider</code>	—	Define provider de interação
<code>SetOutputWriter(w)</code>	<code>io.Writer</code>	—	Redireciona output (Console/SSE)
<code>Run()</code>	—	<code>(Value, error)</code>	Executa bytecode principal , espera background jobs
<code>RunFunction(name, args)</code>	nome, argumentos	<code>(Value, error)</code>	Executa função específica (case-insensitive)

<code>StartJob(funcName, wait, args...)</code>	nome, wait, params	<code>error</code>	Spawn VM isolado em goroutine; se wait=true, bloqueia até fim
<code>execute(instr)</code>	Instruction	<code>error</code>	Dispatch de opcode único (~70 case branches)
<code>newInstance(className, args)</code>	nome classe, args	<code>error</code>	Cria objeto instanciado por classe
<code>callBlockSync(cb, args)</code>	codeblock, args	<code>Value</code>	Avalia codeblock síncrono (ASort/AEval/AScan)
<code>registerMVCModel/view/browse(idx, obj)</code>	—	—	Registra MVC objetos para lookup por Native

Opcodes Implementados no VM (mapa ↔ bytecode)

Opcode	Instrução VM	Opcodes (bytecode)
0	OP_NIL	Push nil
1	OP_TRUE	Push true
2	OP_FALSE	Push false
3	OP_NUMBER	Push number constant
4	OP_STRING	Push string constant
5	OP_DATE	Push date constant
6-7	LOAD/STORE_LOCAL	Stack ↔ local slot
8-9	LOAD/STORE_GLOBAL	Stack ↔ global
10-11	LOAD/STORE_SELF	Object self reference
12-15	NEW_ARRAY, GET, SET, LEN	Array operations
16-18	NEW_OBJECT, GET_PROP, SET_PROP	Object property access
19	CALL_METHOD	obj.Method()
20	NEW_INSTANCE	ClassName():New()
21-22	CALL_FUNC, CALL_NATIVE	Function calls
23-24	RETURN, RETURN_VALUE	Return statements
25	POP	Discard top of stack
26-28	JUMP, JUMP_IF_FALSE, JUMP_IF_TRUE	Control flow
29-32	ADD, SUB, MUL, DIV, MOD, POW, NEG	Arithmetic
31	EQ, NEQ, LT, GT	Comparison
32	LTE, GTE, AND, OR, NOT	Logic/comparison
33	DOLLAR, CONCAT	<code>\$</code> (contains), <code>++</code>
34-35	NEW_CODEBLOCK, EVAL_CODEBLOCK	Code block creation/evaluation
36-37	TRY_BEGIN, TRY_END, THROW, CATCH	Exception handling

38-42	DB_SELECT, DB_SEEK, DB_SKIP, DB_GOTOP, DB_GOBOTTOM, EOF, BOF, FIELD_GET, FIELD_PUT, REC_LOCK, MS_UNLOCK	Database workarea
43-52	OP_MVC_*	MVC scaffolding (registration only)
53	MACRO	Macro substitution <code>&var</code>
54	HALT	Program termination
55	DUP, SWAP	Stack manipulation
56-57	JUMP_IF_FALSE_OR_POP, POP_AND_JUMP	Short-circuit logic
58	NAMED_ARG	Named argument marker
59	FORLOOP_CMP	Step-aware for-loop condition
60-61	LOAD/STORE_UPVAL, LOAD/STORE_DYN, DECL_DYN	Closures & dynamic vars

4.4 Funções Nativas Registradas (~250+)

Todas registradas em `pkg/vm/natives.go` (e `pkg/vm/ui_render.go`, ver 4.4.1) com `v.natives[UPPERCASE_NAME] = fn`.

Saída / Diálogos

Toda saída que pode conter seqüências ANSI (cores, cursor, alt-screen) passa por `stdoutW` (`pkg/vm/natives.go`), um `io.Writer` — `os.Stdout` puro em Linux/macOS, e `go-colorable` no Windows (traduz os escapes ANSI em chamadas da Win32 Console API, já que `cmd.exe`/PowerShell sem `ENABLE_VIRTUAL_TERMINAL_PROCESSING` os imprimiria literalmente). Garante paridade visual entre as 3 plataformas para as mesmas seqüências de bytes.

Função	Args	Retorno	Descrição
<code>CONOUT(...)</code>	variádico	Nil	Imprime no stderr + buffer de output
<code>CONOUTW(...)</code>	variádico	Nil	Same via <code>fmt.Println</code> (sem prefixo <code>stderr</code>)
<code>CONOUTRAW(cText)</code>	string	Nil	Escreve sem newline nem separador — só o 1º arg. Streaming (deltas de LLM token a token) na mesma linha do terminal
<code>MSGINFO(msg, [title])</code>	string, optional title	Nil	Dialog info
<code>MSGSTOP(msg, [title])</code>	string, optional title	Nil	Dialog stop/error
<code>MSGALERT(msg, [title])</code>	string, optional title	Nil	Dialog alert
<code>MSGYESNO(msg, [title])</code>	string, optional title	Bool	Confirmação Yes/No
<code>ALERT(msg)</code>	string	Nil	Dialog alert

<code>FWGETTEXT(prompt, [default], [bIsPassword])</code>	string	string	Input text dialog
<code>FWMENUSELECT(items[], [title])</code>	array	Number	Menu selection (1-based)
<code>CONIN([prompt])</code>	optional string	string Nil	Lê uma linha do stdin. Nil no EOF real (Ctrl+D, pipe esgotado) — distingue de <code>" "</code> (usuário só apertou Enter), pra um REPL poder sair do loop em vez de reimprimir o prompt pra sempre; checar com <code>IsNil()</code>

Quando o stdin é um terminal interativo de verdade (`term.IsTerminal`), `CONIN` entra em raw mode e roda um mini line-editor (`pkg/vm/lineeditor.go`) em vez do antigo `bufio.ReadString('\n')` puro: setas ↑/↓ navegam as últimas 20 linhas digitadas (ignora repetição consecutiva, como `HISTCONTROL=ignoredups` do bash), ←/→/Home/End/Ctrl+A/Ctrl+E movem o cursor, Backspace/Delete editam, Ctrl+U/Ctrl+K apagam até o início/fim da linha, Ctrl+L limpa a tela e Ctrl+C cancela a linha atual (equivale a Enter em branco). Em stdin não-interativo (pipe/redirect, como testes `printf ... | ./binario`) o comportamento continua idêntico ao antigo — sem raw mode. Sem mudança de API: mesma assinatura, mesmo retorno.

4.4.1 TUI / Terminal (`pkg/vm/ui_render.go`)

Primitivas visuais de baixo nível (lipgloss + glamour) para TUIs de terminal escritas em AdvPL/TLPP e compiladas com `advplc build` — estilo opencode/Claude Code (caixas com borda, markdown renderizado, tela alternativa). O programa AdvPL compõe a interface com essas primitivas, em vez de usar widgets prontos (`MSDIALOG` / `FWGETTEXT` / `FWMENUSELECT`), que continuam existindo para diálogos tradicionais).

Função	Args	Retorno	Descrição
<code>UIBOX(cTitle, cBody, cColor, [nWidth])</code>	string×3, optional number	string	Caixa com borda arredondada (lipgloss), título em negrito na 1ª linha. <code>cColor</code> = código ANSI 256 (<code>"39"</code> =ciano, <code>"212"</code> =rosa, <code>"240"</code> =cinza). <code>nWidth</code> omitido/0 = largura automática. Só renderiza a string — quem chama decide se imprime
<code>UISTREAMBOX(cTitle, cBodySoFar, cColor, [nWidth])</code>	string×3, optional number	Nil	Igual <code>UIBox</code> , mas auto-redesenha : apaga com ANSI a caixa anterior (altura rastreada em <code>v.lastBoxLines</code>) antes de imprimir a nova. Chamar a cada delta de um LLM com o texto acumulado até agora (não só o delta) — efeito de "cartão crescendo ao vivo", sem raw-mode de teclado nem redesenho de tela inteira
<code>UISTREAMRESET()</code>	—	Nil	Zera o rastreador de altura do <code>UiStreamBox</code> — chamar ao fechar um turno de streaming, antes do próximo elemento de tela
<code>UIMARKDOWN(cMarkdown, [nWidth])</code>	string, optional number	string	Renderiza markdown (negrito, itálico, listas, blocos de código, títulos) para ANSI via glamour, estilo "dark" fixo (não usa auto-deteção de tema via OSC 11 — pode travar em terminais/multiplexers que não respondem a essa query, mesma cautela já documentada para lipgloss em <code>pkg/ui/terminal.go</code>). <code>nWidth</code> padrão 80. Em erro de parse devolve <code>cMarkdown</code> sem alteração, nunca falha
<code>UIALTSCREENENTER()</code>	—	Nil	Entra na tela alternativa do terminal (mesmo buffer que vim/less/htop usam) — a saída normal do shell fica preservada. Instala handler de Ctrl+C (<code>os.Interrupt</code> , portátil) que restaura a tela antes de encerrar

<code>UIALTSCREENEXIT()</code>	—	Nil	Sai da tela alternativa, restaura o conteúdo normal — chamar na saída normal do programa
<code>UITERMWIDTH([nDefault])</code>	optional number	number	Largura do terminal em colunas. <code>nDefault</code> (padrão 80) é usado quando stdout não é um tty real (pipe/redirecionamento), onde <code>term.GetSize</code> falha

Manipulação de Strings

Função	Args	Retorno	Descrição
<code>ALLTRIM(str)</code>	string	string	Remove whitespace ambos lados
<code>LTRIM(str)</code>	string	string	Remove left whitespace
<code>RTRIM(str)</code>	string	string	Remove right whitespace
<code>TRIM(str)</code>	string	string	Alias para RTRIM
<code>STR(num, [width], decimals)</code>	numeric	string	Número formatado como string
<code>STRTRAN(str, search, replace)</code>	3 strings	string	Substituição global
<code>STRZERO(num, size)</code>	numeric, int	string	Zero-padded string
<code>SUBSTR(str, start, [len])</code>	string	string	Substring (1-based)
<code>STUFF(str, start, count, repl)</code>	4 strings	string	Substitui substring
<code>`LEN(str</code>	array)`	string or array	Number
<code>AT(search, str)</code>	2 strings	Number	Posição 1-based (0=não encontrado)
<code>RAT(search, str)</code>	2 strings	Number	Última posição
<code>ATC(search, str)</code>	2 strings	Number	Case insensitive AT
<code>RATC(search, str)</code>	2 strings	Number	Case insensitive RAT
<code>UPPER(str)</code>	string	string	Maiúsculas
<code>LOWER(str)</code>	string	string	Minúsculas
<code>PADC(str, size, [pad])</code>	3 strings	string	Central-pad
<code>PADL(str, size, [pad])</code>	3 strings	string	Left-pad
<code>PADR(str, size, [pad])</code>	3 strings	string	Right-pad
<code>CHR(code)</code>	number	string	Caractere pelo código
<code>ASC(str)</code>	string	number	Código do caractere
<code>VAL(str)</code>	string	number	String → número
<code>CVALTOCHAR(val)</code>	any	string	Valor para string
<code>EMPTY(val)</code>	any	bool	.T. se vazio/nulo
<code>SPACE(n)</code>	number	string	n espaços
<code>REPLICATE(str, n)</code>	2 args	string	Repete n vezes
<code>REPLICA(str, n)</code>	2 args	string	Alias para REPLICATE

<code>STRtokARR(str, delim)</code>	2 strings	array	Split string → array
<code>LEFT(str, count)</code>	2 args	string	n char da esquerda
<code>RIGHT(str, count)</code>	2 args	string	n char da direita
<code>CAPSLock(str)</code>	string	string	Capitalize first letter
<code>PROPER(str)</code>	string	string	Title Case
<code>GETWORDNUM(str, num, [delim])</code>	string	string	Palavra n-ésima
<code>WORDS(str, [delim])</code>	string	number	Contagem de palavras
<code>FILENOEXT(path)</code>	string	string	Nome sem extensão
<code>FILEEXT(path)</code>	string	string	Extensão
<code>FILENAME(path)</code>	string	string	Nome do arquivo
<code>FILEPATH(path)</code>	string	string	Caminho do arquivo
<code>FILEDIR(path)</code>	string	string	Diretório

Numéricas / Matemática

Função	Args	Retorno	Descrição
<code>ABS(n)</code>	number	number	Valor absoluto
<code>INT(n)</code>	number	number	Trunca para inteiro
<code>ROUND(n, decimals)</code>	2 args	number	Arredonda
<code>NOROUND(n, decimals)</code>	2 args	number	Trunca decimais
<code>CEILING/FLOOR(n)</code>	number	number	Teto/chão
<code>MOD(a, b)</code>	2 numbers	number	Resto da divisão
<code>MAX(...)</code>	variádico	number	Máximo de args
<code>MIN(...)</code>	variádico	number	Mínimo de args
<code>SQRT(n)</code>	number	number	Raiz quadrada
<code>EXP(n)</code>	number	number	e^x
<code>LOG(n)</code>	number	number	Log natural
<code>RANDOM([max])</code>	optional number	number	Random 1..max
<code>SIGN(n)</code>	number	number	-1/0/1
<code>POWER(base, exp)</code>	2 numbers	number	x^y
<code>PI()</code>	—	number	Constante π
<code>SIN/COS/TAN(rads)</code>	number	number	Trigonométricas
<code>ASIN/ACOS/ATAN(rads)</code>	number	number	Trig inversas
<code>DEG2RAD(deg)</code>	number	number	Graus → radianos

<code>RAD2DEG(rad)</code>	number	number	Radianos → graus
---------------------------	--------	--------	------------------

Data / Tempo

Função	Args	Retorno	Descrição
<code>DATE()</code>	—	Date	Data atual
<code>DAY(date)</code>	date	number	Dia
<code>MONTH(date)</code>	date	number	Mês
<code>YEAR(date)</code>	date	number	Ano
<code>CMONTH(date)</code>	date	string	Nome do mês
<code>CDOW(date)</code>	date	string	Nome do dia da semana
<code>DOW(date)</code>	date	number	1=Domingo...7=Sábado
<code>TIME()</code>	—	string	"HH:MM:SS"
<code>SECONDS()</code>	—	number	Timestamp Unix
<code>CTOD(str)</code>	"DD/MM/YYYY"	Date	String → date
<code>DTOS(date)</code>	date	string	Date → "YYYYMMDD"
<code>DTOC(date)</code>	date	string	Date → "DD/MM/YYYY"
<code>STOD(str)</code>	"YYYYMMDD"	date	String → date
<code>ELAPTIME(t1, t2)</code>	2 times	number	Diferença em segundos
<code>CTOT(str)</code>	"HH:MM:SS"	date	Time → date
<code>TTOC(d/t)</code>	date/time	string	→ "HH:MM:SS"
<code>FW8601TODATE(str)</code>	RFC3339	date	ISO → Protheus date
<code>FWDATET08601(d)</code>	date	string	Protheus → RFC3339
<code>SETDATE(fmt)</code>	string	—	Formato data (stub)
<code>SETCENT(n)</code>	number	—	Configurar século (stub)
<code>ARRAY(dims...)</code>	variadic	array	Array multidimensional de Nils

Transforms

Função	Args	Retorno	Descrição
<code>TRANSFORM(val, mask)</code>	any, string	string	Aplica máscara de formatação

Checks de Caractere

Função	Args	Retorno	Descrição
<code>ISDIGIT(str)</code>	string	bool	Só dígitos?
<code>ISALPHA(str)</code>	string	bool	Só letras?

<code>ISLOWER(str)</code>	string	bool	Minúsculas?
<code>ISUPPER(str)</code>	string	bool	Maiúsculas?

Arrays

Função	Args	Retorno	Descrição
<code>AADD(array, val)</code>	2 args	value	Append (return val)
<code>ASIZE(array, size)</code>	2 args	Nil	Redimensiona (padra com NIL)
<code>ASCAN(array, search[, start, count])</code>	variadic	number	Busca (1-based, 0=não achou)
<code>ADEL(array, idx)</code>	2 args	Nil	Delete por índice
<code>AINS(array, idx)</code>	2 args	Nil	Insert NIL shift-right
<code>ALEN(array)</code>	1 arg	number	Length
<code>ACLONE(array)</code>	1 arg	array	Shallow copy
<code>AFILL(array, val)</code>	2 args	Nil	Preenche array
<code>ASORT(array [,start,count,block])</code>	variadic	array	Ordenação (com comparator opcional)
<code>AEVAL(array, block[,start,count])</code>	variadic	array	Avalia bloco para cada elemento

Lógica / Tipagem

Função	Args	Retorno	Descrição
<code>IIF(cond, true_val, false_val)</code>	3 args	value	Condicional ternário (short-circuit)
<code>IF(cond, true_val, false_val)</code>	3 args	value	Alias para IIF
<code>VALTYPE(val)</code>	any	string	Tipo: "N"/"C"/"L"/"A"/"B"/"O"/"U"/"D"
<code>TYPE(expr)</code>	identifier	string	Tipo de identificador
<code>ISNIL(val)</code>	any	bool	É NIL?

Tratamento de Erros

Função	Args	Retorno	Descrição
<code>USEREXCEPTION(msg)</code>	string	—	Lança exception
<code>THROW(msg)</code>	string	—	Alias para USEREXCEPTION

Sistema / Misc

Função	Args	Retorno	Descrição
<code>FINDFUNCTION(name)</code>	string	bool	Função existe?
<code>STARTJOB(funcName, env, wait, params...)</code>	variadic	bool	Background job
<code>SLEEP(ms)</code>	number	Nil	Sleep em milissegundos
<code>PROCNAME()</code>	—	string	Nome do proc (stub: "")

<code>PROCLINE()</code>	—	number	Linha do proc (stub: 0)
<code>GETMV(key, default)</code>	2 args	value	Get MV parameter (stub: returns 2nd arg)
<code>GETNEWPAR(key)</code>	string	string	Nova parâmetro (stub)
<code>GETENV(name, [default])</code>	string	string	Environment variable
<code>FWHASH(text)</code>	string	string	SHA-256 hex digest
<code>FILE(path)</code>	string	bool	Arquivo existe?
<code>MAKEDIR(path)</code>	string	Nil	Criar diretório (stub)
<code>CURDIR()</code>	—	string	"./"
<code>GETSRVNAME()</code>	—	string	"localhost"
<code>FWHTTPENCODE(str)</code>	string	string	URL percent encoding
<code>FWURIDECODE(str)</code>	string	string	URI decode (pass-through)
<code>FWHTTPGET(cURL, [cCert, cPass])</code>	string, opcionais	number	Requisição GET, retorna o status HTTP
<code>FWHTTPPOST(cURL, cBody, cContentType, [cCert, cPass])</code>	string×3, opcionais	number	Requisição POST
<code>FWHTTPPUT(cURL, cBody, cContentType, [cCert, cPass])</code>	string×3, opcionais	number	Requisição PUT
<code>FWHTTPPATCH(cURL, cBody, cContentType, [cCert, cPass])</code>	string×3, opcionais	number	Requisição PATCH
<code>FWHTTPDELETE(cURL, [cCert, cPass])</code>	string, opcionais	number	Requisição DELETE
<code>FWHTTPBODY()</code>	—	string	Corpo da resposta da última requisição
<code>FWHTTPSTATUS()</code>	—	number	Status HTTP da última requisição
<code>FWHTTPERROR()</code>	—	string	Mensagem de erro da última requisição (se houver)
<code>FWHTTPTIMEOUT(nSeconds)</code>	number	Nil	Define o timeout (segundos) aplicado a toda requisição <code>FWHTTPGET/POST/PUT/PATCH/DELETE</code> seguinte, até ser trocado de novo. <code>nSeconds <= 0</code> restaura o default de 30s
<code>FWHTTPHEADER(cName, cValue)</code>	string, string	Nil	Registra um header customizado (ex.: <code>Authorization: Bearer ...</code>) aplicado a toda requisição seguinte, até <code>FWHTTPCLEARHEADERS()</code>
<code>FWHTTPCLEARHEADERS()</code>	—	Nil	Remove todos os headers customizados definidos via <code>FWHTTPHEADER</code>

<code>FWLOADSM0()</code>	—	bool	.T. stub
<code>FWJOINFILIAL(field, alias)</code>	2 strings	string	Concat filial
<code>FWRESTAREA/FWGETAREA/FWAPPSTACK/FWCALLAPP</code>	—	Nil/string	Stubs
<code>FWLIBVERSION()</code>	—	string	"1.0.0"
<code>FWLISTBRANCHES()</code>	—	array	Empty array
<code>FWCLEARHLP()</code>	—	Nil	Clear help (stub)
<code>FWMSGRUN(cCmd, lWait)</code>	variadic	string	MsgRun (stub)
<code>FWMONITORMSG(msg, level)</code>	variadic	Nil	Monitor msg (stub)
<code>MPISSMART()</code>	—	bool	.F. stub
<code>MPUSERHASACCESS()</code>	—	bool	.T. stub
<code>MPCRANUMS()</code>	—	string	"000001" stub
<code>MPDOCPATH()</code>	—	string	".,/"
<code>MPDOCVIEW/MPBINVIEW/MPEXPCHK/MSDOCUMENT/MSMULTDIR</code>	—	Nil/array	Viewer stubs
<code>CHANGEQUERY(query)</code>	string	Nil	FW exec SQL (stub)
<code>CHKADVPLSYNTAX()</code>	—	bool	Syntax check (stub)
<code>FILLGETDADOS()</code>	—	Nil	DB fill (stub)
<code>FWEXECLocaliz()</code>	—	Nil	Localization exec (stub)
<code>FWEXISTLOCALIZ()</code>	—	bool	Localization exists (stub)
<code>FWQTTOCHR(qt)</code>	number	string	Quote-to-char
<code>FWREBUILDINDEX()</code>	—	bool	Rebuild index (stub)
<code>FWRULESDB()</code>	—	bool	Rules DB (stub)
<code>FWGRPPRIVDB()</code>	—	bool	Group priv DB (stub)
<code>FWPDCANUSE(username)</code>	string	bool	Auth can use (stub)
<code>FWPDLOGUSER(username)</code>	string	bool	Auth log user (stub)
<code>FWPUTSX5(key, val)</code>	2 args	—	Put SX5 (stub)
<code>FWX2CHAVE(field)</code>	string	bool	SX2 unique key (stub)
<code>FWX2UNICO(field)</code>	string	bool	SX2 unique (stub)
<code>FWX3TITULO(field)</code>	string	string	SX3 title (stub)
<code>FWUSREMP(username)</code>	string	string	User emp (stub)
<code>FWVLDVINC(cVar)</code>	string	string	Validate variant (stub)
<code>PESQBRW()</code>	—	Nil	Browse search (stub)
<code>MARKBROW()</code>	—	Nil	Mark browse (stub)
<code>MAKESQLEXPR(alias, fields)</code>	variadic	string	SQL expression (stub)

<code>MAYIUSECODE()</code>	—	bool	Use code (stub)
<code>RESTINTER(savePath)</code>	string	Nil	REST inter (stub)
<code>SAVEINTER(fileName)</code>	string	Nil	Save inter (stub)
<code>PUTSX1HELP(field, help)</code>	2 strings	—	SX1 help (stub)
<code>OLE_CREATELINK(progId)</code>	string	object	OLE create (stub)
<code>PROCESSA()</code>	—	Nil	Process A (stub)
<code>MENUDEF()</code>	—	Nil	Menu def (stub)
<code>I18N(key)</code>	string	string	i18n translation (stub)
<code>WSADVVALUE(value)</code>	any	value	WS adv value (stub)
<code>GETNAMES(obj)</code>	JsonObject	array	Lista propriedades do objeto
<code>HELP()</code>	—	Nil	Help (stub)
<code>FREEOBJ/FWFREEOBJ(obj)</code>	object	Nil	Free object (stub/no-op)
<code>FWALIASINDIC()</code>	—	bool	.F. stub
<code>FWMODEACCESS()</code>	—	number	1 stub
<code>FWHASACCMODE()</code>	—	bool	.T. stub
<code>USRRETNAME()</code>	—	string	Username stub
<code>MSRETPATH()</code>	—	string	"/," stub
<code>WAITRUN(cmd)</code>	string	number	Executa shell command, retorna exit code
<code>PROCRUN(cPath, aArgs, b0nStdoutLine, [b0nStderrLine])</code>	string, array, codeblock×2	number	Executa <code>cPath</code> com <code>aArgs</code> (sem shell, stdin fechado). Para cada linha de stdout chama <code>b0nStdoutLine(cLinha)</code> sincronamente — o AdvPL pode desenhar direto na tela, ideal pra TUIs consumindo NDJSON de um processo filho em streaming (ex.: CLI de um LLM). <code>b0nStderrLine</code> opcional (stderr descartado se omitido). Bloqueia até o processo terminar; retorna o exit code, ou -1 se não iniciou

Funções Database (Workarea)

Função	Args	Retorno	Descrição
<code>DBSELECTAREA(alias)</code>	string	Nil	Seleciona work area
<code>DBSEEK(key)</code>	any	Bool	Seek por chave
<code>DBSKIP([count])</code>	optional number	Nil	Skip N registros
<code>DBGOTOP()</code>	—	Nil	Vai ao primeiro

<code>DBGOBOTTOM()</code>	—	Nil	Vai ao último
<code>EOF()</code>	—	Bool	No fim?
<code>BOF()</code>	—	Bool	No início?
<code>RELOCK()</code>	—	Bool	Lock record
<code>MSUNLOCK()</code>	—	Nil	Unlock
<code>RECCOUNT()</code>	—	Number	Total registros
<code>RECNO()</code>	—	Number	Nº registro atual
<code>DBCLOSEAREA()</code>	—	Nil	Close area (stub)
<code>DBSETORDER(nOrd)</code>	number	Nil	Set order (stub)
<code>DBSETFILTER(exp, exp2)</code>	variadic	Nil	Set filter (stub)
<code>DBCLEARFILTER()</code>	—	Nil	Clear filter (stub)
<code>DBAPPEND()</code>	—	Nil	Append record
<code>DBDELETE()</code>	—	Nil	Mark delete
<code>DBCOMMIT()</code>	—	Nil	Commit transaction
<code>SELECT()</code>	—	Number	Número da área
<code>ALIAS()</code>	—	String	Alias atual
<code>GETAREA([alias])</code>	optional	String	Salva área corrente
<code>RESTAREA(cAlias)</code>	string	—	Restaura área
<code>RETSQLNAME(alias)</code>	string	string	Nome físico tabela
<code>USED()</code>	—	Bool	.F. stub
<code>FIELDGET(field)</code>	—	Nil	Read field (stub)
<code>FIELDPUT(field, val)</code>	—	Nil	Write field (stub)
<code>FIELDPOS(field)</code>	string	Number	Position of field
<code>FIELDNAME()</code>	—	String	(stub)
<code>XFILIAL(alias)</code>	optional string	string	Filial filter
<code>TCSQLEXEC(query)</code>	string	Bool	Raw SQL execute
<code>TCSQLQUERY(query)</code>	string	array	Raw SQL query → JsonObject[]

MVC Functions

Função	Args	Retorno	Descrição
<code>FWFORMMODEL(name)</code>	string	Object	Cria FWFormModel
<code>FWFORMVIEW(name, model)</code>	2 strings	Object	Cria FWFormView
<code>FWFORMBROWSE(name, model)</code>	2 strings	Object	Cria FWFormBrowse
<code>FWFORMSTRUCT()</code>	—	Object	Form struct (stub)

<code>FWMBROWSE()</code>	—	Object	FWMBrowse object
<code>VIEWDEF()</code>	—	Nil	Scaffolding stub
<code>AXCADAstro()</code>	—	Nil	Cadastro stub

JSON

Função	Args	Retorno	Descrição
<code>JSONOBJECT()</code>	—	Object	Cria JsonObject

Métodos JsonObject

Acesso a chave é por colchete (`oJ["chave"]` , `oJ["a"]["b"]`), não por dois-pontos —

`oJ:chave` não resolve propriedades dinâmicas (só campos declarados de classe).

Método	Args	Retorno	Descrição
<code>oJ:FromJson(cJson)</code>	string	Bool	Parser real (<code>encoding/json</code> da stdlib) — antes era stub que sempre devolvia Nil sem parsear nada. Objetos aninhados viram <code>JsonObject</code> , arrays viram <code>Array</code> 1-based, <code>null</code> → Nil. <code>.F.</code> se <code>cJson</code> não parsear como objeto (JSON inválido, ou top-level array/escalar) — nunca falha/lança erro
<code>oJ:ToString()</code> / <code>oJ:ToJson()</code>	—	string	Serializa de volta para JSON
<code>GetNames(oJ)</code>	JsonObject	array	Lista as propriedades, na ordem de inserção

Math/Stat Extended

Função	Args	Retorno	Descrição
<code>CEIL(n)</code>	number	number	Ceiling
<code>COSH/SINH/TANH(x)</code>	number	number	Hyperbolic
<code>FACT(n)</code>	number	number	Fatorial
<code>GCD(a, b)</code>	2 numbers	number	MDC
<code>LCM(a, b)</code>	2 numbers	number	MMC
<code>MEAN(array)</code>	array	number	Média aritmética
<code>VARIANCE(array)</code>	array	number	Variância amostral
<code>STDDEV(array)</code>	array	number	Desvio padrão
<code>MEDIAN(array)</code>	array	number	Mediana
<code>LINREG(xArr, yArr)</code>	2 arrays	[intercept, slope]	Regressão linear
<code>INTERP(xArr, yArr, x)</code>	3 args	number	Interpolação linear
<code>ATAN2(y, x)</code>	2 numbers	number	Arctan 4 quadrantes
<code>LOG10(n)</code>	number	number	Log10
<code>POW(base, exp)</code>	2 numbers	number	Power

<code>MATVECTERN(matrix, ternaryVec)</code>	2 args	array	Matrix-vector mul (sparse BLAS)
<code>FIT(block, epochs)</code>	variadic	value	Treinamento loop

4.5 Sistema de Diálogo (MsDialog)

Arquivo: `pkg/vm/dialog.go`

Função Nativa	Args	Retorno	Descrição
<code>MSDIALOG(x1,y1,x2,y2,title)</code>	5 args	Object	Cria objeto MsDialog
<code>AT_SAY(row,col,text,...)</code>	variadic	Nil	Registra SAY control
<code>AT_GET(row,col,var,val,picture,...)</code>	variadic	Nil	Registra GET control
<code>AT_BUTTON(row,col,text,action,...)</code>	variadic	Nil	Registra BUTTON control
<code>AT_BOX()</code>	—	Nil	Decorativo (no-op)
<code>ACTIVATE_MSDIALOG(dialogObj, [initBlock])</code>	variadic	Nil	Executa ciclo do diálogo

Métodos no Objeto MsDialog

`NEW()`, `ACTIVATE()`, `END()`, `CLOSE()`, `DEACTIVATE()`, `SETTITLE()`, `SETPOS()`, `SETSIZE()`, `ADDSAY()`, `ADDGET()`, `ADDSTRUCTGET()`, `ADDSTRUCT()`, `ADDSBUTTON()`, `ADDIMAGE()`, `ADDWINDOW()`

4.6 Browse (FWMBrowse Auto-CRUD)

Arquivo: `pkg/vm/browse.go`

Função Nativa	Args	Retorno	Descrição
<code>FWMBROWSE()</code>	—	Object	Cria FWMBrowse

Métodos FWMBrowse

Método	Args	Retorno	Descrição
<code>NEW()</code>	—	—	Constructor
<code>SETALIAS(alias)</code>	string	—	Sets table alias
<code>SETTITLE(title)</code>	string	—	Sets dialog title
<code>ACTIVATE()</code>	—	—	Full CRUD cycle
<code>DEACTIVATE()</code>	—	—	Destroys dialog
<code>DESTROY()</code>	—	—	Cleanup

Fluxo `runBrowse()`

1. Detecta engine SQL + alias
2. Queries PRAGMA table_info + SX3 dictionary → columns metadata
3. SELECT rowid AS browse_recno_ WHERE D_E_L_E_T_=' ' → row items
4. Aguarda ação do usuário: save (INSERT/UPDATE), delete (soft-delete D_E_L_E_T_='*'), close
5. Retorna ao passo 3 para refresh

4.7 Grid Process (FWGridProcess)

Arquivo: `pkg/vm/grid.go`

Função Nativa	Args	Retorno	Descrição
<code>FWGRIDPROCESS()</code>	—	Object	Creates grid process

Métodos FWGridProcess

Método	Args	Retorno	Descrição
<code>NEW(cFunName, cTitle, cDesc, bProcess, cPerg, cGrid, lSaveLog)</code>	variadic	—	Constructor
<code>SETTHREADGRID(nThreads)</code>	number	—	Thread pool size
<code>SETMAXTHREADGRID(max)</code>	number	—	Max concurrent
<code>ACTIVATE()</code> / <code>EXECUTE()</code>	—	—	Executa bProcess síncrono
<code>CALLEXCUTE(cb)</code>	codeblock	—	Dispatch async worker
<code>STOPEXCUTE()</code>	—	—	Stops all workers
<code>ISFINISHED()</code>	—	bool	Are all done?
<code>SETABORT(lAbort)</code>	bool	—	Set abort flag
<code>SETAFTEREXECUTE(cb)</code>	codeblock	—	Post-exec callback
<code>SETMETERS(nMeters)</code>	number	—	Number progress meters
<code>SETMAXMETER(idx, max)</code>	2 args	—	Meter max value
<code>SETINCMETER(idx, inc)</code>	2 args	—	Increment meter
<code>SAVELOG()</code>	—	Nil	Save log
<code>GETLASTLOG()</code>	—	string	Last log message
<code>SETNOPARAM()</code>	—	—	Disable parameters
<code>DEACTIVATE()</code>	—	—	Destroy

4.8 Macro Substitution (&)

Arquivo: `pkg/vm/macro.go`

Função	Descrição
<code>evalMacroString(src string)</code>	Pipeline completo: lex → parse → compile → executa string como expressão AdvPL em frame isolado. Variáveis dynEnv (Private/Public) são compartilhadas com caller. Retornar valor da expressão após restore do frame.

Suporte: `&ident`, `&(expression)`, `K2&cSuf` (macro-computed variable names).

4.9 Debugger

Arquivo: `pkg/vm/debug.go` + `pkg/dap/`

Tipos debugger (`pkg/vm/`):

Tipo	Campos	Descrição
<code>StepMode</code>	<code>StepNone</code> , <code>StepOver</code> , <code>StepIn</code> , <code>StepOut</code>	Modos de step
<code>Debugger</code>	<code>breakpoints</code> , <code>pauseRequested</code> , <code>resumeChan</code> , <code>OnStop</code>	Debugger instance

<code>StackFrameInfo</code>	<code>Name string, Line int</code>	Info de frame na stack
<code>VarInfo</code>	<code>Name, Type, Value</code>	Variável local info

Método	Descrição
<code>NewDebugger()</code>	Cria debugger
<code>v.AttachDebugger(d)</code>	Liga debugger à VM
<code>d.SetBreakpoints(lines)</code>	Breakpoints por linha
<code>d.SetStopOnEntry(v)</code>	Pausa na entrada de função
<code>d.checkBreak(v, instr)</code>	Per instruction — decide se pausar
<code>d.pause(reason, line)</code>	Para execução, chama OnStop
<code>d.Continue()/Next()/StepIn()/StepOut()</code>	Resume execution
<code>d.RequestPause()</code>	Request pause imediato
<code>v.DebugStackFrames()</code>	<code>[]StackFrameInfo</code> — call stack
<code>v.DebugLocals(frameIndex)</code>	<code>[]VarInfo</code> — variáveis locais de frame

DAP Server (`pkg/dap/`):

Função	Descrição
<code>NewConn(r, w)</code>	stdio connection with Content-Length framing
<code>NewServer(conn, compile, attach)</code>	Launch mode — compila e executa fonte
<code>NewAttachServer(conn, sourcePath)</code>	Attach mode — depura sessions browser
<code>OfferSession(v)</code>	Oferece VM para debugging (attach mode)
<code>Run()</code>	Event loop: ReadMessage → handle → repeat

Comandos DAP implementados: `initialize`, `launch`, `attach`, `setBreakpoints`, `configurationDone`, `threads`, `stackTrace`, `scopes`, `variables`, `continue`, `next`, `stepIn`, `stepOut`, `pause`, `disconnect`, `terminate`

4.10 Tensor (Operações com Tensores)

Arquivo: `pkg/tensor/tensor.go` + `ops.go` + `ops64.go` + `linalg.go`

Tipos:

Tipo	Campos	Descrição
<code>DType</code>	—	<code>Float32</code> (default), <code>Float64</code>
<code>Tensor</code>	<code>Data []float32</code> , <code>Shape []int</code> , <code>Data64 []float64</code> , <code>DType DType</code>	Dense row-major tensor

Constructors

Função	Parâmetros	Retorno
<code>New(shape []int)</code>	<code>dims</code>	zero-filled float32

<code>FromData(data []float32, shape []int)</code>	dados + shape	tensor com dados
<code>Rand(shape []int, scale float32)</code>	dims, range	random [-scale, scale]
<code>NewDType(shape, dt)</code>	dims + dtype	tensor typed zero
<code>FromData64(data, shape)</code>	f64 data + shape	float64 tensor
<code>Prod(shape)</code>	dims	produto de todos (= total elements)

Métodos Tensor

Método	Args	Retorno	Descrição
<code>Size()</code>	—	int	Total elementos
<code>Get(i int)</code>	index	float64	Elemento flat
<code>Set(i int, v float64)</code>	index, val	—	Escreve elemento
<code>At(idx []int)</code>	multi-dim	(float32, error)	Elemento por dimensões
<code>SetAt(idx []int, val)</code>	indices, val	error	Escreve por dimensões
<code>Offset(idx []int)</code>	indices	(int, error)	Flat offset
<code>AsDType(dt)</code>	target dtype	*Tensor	Copy com dtype
<code>Add/Sub/Mul/Div(b)</code>	other tensor	(*Tensor, error)	Element-wise binary
<code>AddScalar/MulScalar(s)</code>	scalar	*Tensor	Element-wise scalar
<code>Exp/Log/Sqrt/Tanh/Relu/Sigmoid/Gelu</code>	—	*Tensor	Activation element-wise
<code>Softmax(axis)</code>	0 or 1	(*Tensor, error)	Softmax (stable)
<code>MatMul(b)</code>	other	(*Tensor, error)	Matrix multiplication
<code>Transpose()</code>	—	(*Tensor, error)	2D transpose
<code>Reshape(shape)</code>	new shape	(*Tensor, error)	New shape same data
<code>IndexRows(idx)</code>	row indices	(*Tensor, error)	2D row subset

Reductions

Método	Args	Retorno	Descrição
<code>SumAll/MeanAll/MaxAll</code>	—	float32	Global reduction
<code>ArgmaxAll</code>	—	int	Index do máximo
<code>SumAxis/MeanAxis/MaxAxis/ArgmaxAxis</code>	axis 0/1	(*Tensor, error)	Reduction by axis

Linear Algebra (`linalg.go`)

Método	Args	Retorno	Descrição
<code>Dot(b)</code>	vector	(float64, error)	Dot product
<code>Norm()</code>	—	float64	Frobenius norm (L2)
<code>Det()</code>	—	(float64, error)	Determinant (2D square)

<code>Inv()</code>	—	(*Tensor, error)	Inverse via LU
<code>Solve(b)</code>	rhs [n]	(*Tensor, error)	Ax=b solution
<code>QR()</code>	—	(Q, R, error)	Householder QR
<code>EigSym()</code>	—	(vals, vecs, error)	Symmetric eigenvalues/eigenvectors
<code>Eig()</code>	—	(real, imag, error)	Full eigenvalues (complex)

4.11 Autograd (Diferenciação Automática)

Arquivo: `pkg/autograd/variable.go` + `sgd.go` + `adam.go` + `ops.go`

Tipo Principal: `Variable`

Campo	Tipo	Descrição
<code>Value</code>	<code>*tensor.Tensor</code>	Valor atual
<code>Grad</code>	<code>*tensor.Tensor</code>	Gradiente acumulado
<code>parents</code>	<code>[]*Variable</code>	Tape: quem gerou este
<code>backward</code>	<code>func()</code>	Closure de gradiente

Constructors

Função	Parâmetros	Retorno
<code>NewLeaf(t *tensor.Tensor)</code>	tensor	Leaf variable (sem parents)

Métodos de Autograd (gradient tape)

Método	Args	Retorno	Descrição
<code>MatMul(b)</code>	*Variable	*Variable	Forward + backward: $dA=dY \cdot B^T$, $dB=A^T \cdot dY$
<code>Add(b)</code>	*Variable	*Variable	Sum com broadcast gradient
<code>Mul(b)</code>	*Variable	*Variable	Hadamard: $dA=dY \odot B$, $dB=dY \odot A$
<code>Relu()</code>	—	*Variable	$\text{relu}(x)$: $dA=dY \odot (x>0)$
<code>Sum()</code>	—	*Variable	Scalar: broadcast ones
<code>Mean()</code>	—	*Variable	Scalar: dY/N broadcast
<code>Tanh()</code>	—	*Variable	$dA=dY \odot (1-\tanh^2)$
<code>Sigmoid()</code>	—	*Variable	$dA=dY \odot \sigma(1-\sigma)$
<code>Gelu()</code>	—	*Variable	GELU tanh approximation
<code>IndexRows(idx)</code>	[]int	*Variable	Scatter-add gradient back
<code>Reshape(shape)</code>	[]int	*Variable	Gradient reshape to original
<code>SoftmaxCE(targets)</code>	[]int	*Variable	Cross-entropy loss
<code>MSE(target)</code>	*Variable	*Variable	Mean squared error
<code>Backward()</code>	—	error	Topological reverse pass (scalar loss required)

--	--	--	--

Optimizers

SGD:

Método	Args	Retorno	Descrição
<code>NewSGD(params, lr)</code>	<code>[]*Variable, float32</code>	<code>*SGD</code>	Standard SGD
<code>Step()</code>	—	—	$p := p - lr \cdot grad$ (in-place)
<code>ZeroGrad()</code>	—	—	Reset all gradients

Adam:

Método	Args	Retorno	Descrição
<code>NewAdam(params, lr)</code>	<code>[]*Variable, float32</code>	<code>*Adam</code>	Adam (b1=0.9, b2=0.999, eps=1e-8)
<code>Step()</code>	—	—	Bias-corrected update
<code>ZeroGrad()</code>	—	—	Reset gradients + moment buffers

4.12 LLM (Inferência GGUF)

Arquivo: `pkg/llm/model.go` + `tokenizer.go` + `gguf.go` + `i2s.go` + `sampling.go` + `simd_amd64.go`

GGUF File Operations

Função	Parâmetros	Retorno	Descrição
<code>Open(path)</code>	filepath	<code>(*File, error)</code>	Abre GGUF (lazy tensor loads)
<code>Close()</code>	—	error	Fecha arquivo
<code>Tensor(name)</code>	name	<code>(*Tensor, bool)</code>	Lookup tensor descriptor
<code>TensorData(name)</code>	name	<code>[]byte, error)</code>	Full tensor data
<code>TensorRange(name, off, len)</code>	name, offset, length	<code>[]byte, error)</code>	Partial read
<code>Uint32/Float32/String/StringArray/Int32Array(key)</code>	KV key	value + found	Metadata KV queries

Model Loading

Função	Parâmetros	Retorno	Descrição
<code>LoadModel(path)</code>	GGUF path	<code>(*Model, error)</code>	Loads arch=llama only; ~file-size RAM
<code>Close()</code>	—	error	Closes underlying GGUF file

Context / Inference

Função	Parâmetros	Retorno	Descrição
<code>NewContext(m)</code>	<code>*Model</code>	<code>*Context</code>	Empty context (KV cache)
<code>Forward(token)</code>	int32 ID	<code>[]float32, error)</code>	Single-token forward pass. Updates KV cache.

Tokenizer

Função	Parâmetros	Retorno	Descrição
<code>NewTokenizer(g)</code>	*File	<code>(*Tokenizer, error)</code>	Parses GPT-2 BPE vocab from GGUF
<code>BOS()</code>	—	int32	Beginning of sequence ID
<code>EOS()</code>	—	int32	End of sequence ID
<code>Encode(text)</code>	string	[]int32	Text → token IDs (BPE)
<code>Decode(ids)</code>	[]int32	string	Token IDs → text (BPE)

Sampling

Função	Parâmetros	Retorno	Descrição
<code>Greedy(logits)</code>	[]float32	int32	Argmax (best token)
<code>Sample(logits, cfg, rng)</code>	logits, config, rand	int32	Nucleus sampling; temp≤0 → greedy

Kernel Functions (internal)

Função	Descrição
<code>MatMulI2S(w, x)</code>	Quantized ternary matmul (I2_S weights)
<code>MatMulF16(weightF16, nRows, nIn, x)</code>	F16 matmul (SIMD AVX2 accelerated on amd64)
<code>RMSNorm(x, weight, eps)</code>	RMS normalization
<code>RoPE(x, headDim, pos, ropeDims, freqBase)</code>	Rotary position embedding (in-place)
<code>SwiGLU(gate, up)</code>	SiLU(gate) * up — LLaMA FFN activation
<code>Softmax(x)</code>	Normalizes logits in-place
<code>EmbedRow(g, name, row, dim)</code>	Lazy F16 row decode from GGUF
<code>quantizeI8S(x)</code>	Absmax quantization (±127)
<code>Float16ToFloat32(h)</code>	Half-float → float32
<code>DecodeF16Row(raw, n)</code>	F16 bytes → float32
<code>parallelRows(nRows, fn)</code>	Goroutines-based parallel processing

SIMD Acceleration (amd64)

Variável/Função	Descrição
<code>hasAVX2</code>	Runtime CPUID detection
<code>hasF16CFMA</code>	F16+CFMA detection
<code>dotI2SBlocksAVX2()</code>	AVX2 ternary matmul kernel (Plan9 assembly)
<code>dotF16BlocksAVX2()</code>	VCVTPH2PS+FMA F16 dot (assembly)
<code>cpuuid()</code>	CPUID instruction (assembly)

4.13 TMailMessage (E-mail SMTP)

Arquivo: `pkg/vm/mail_native.go`

Função Nativa	Args	Retorno	Descrição
<code>TMAILMESSAGE()</code>	—	Object	Create TMailMessage class
<code>NEW()</code>	—	self	Constructor
<code>SETSERVER(server, [port])</code>	string, optional int	Nil	SMTP server + port (default 587)
<code>SETAUTH(user, pass)</code>	2 strings	Nil	Plain auth credentials
<code>SETFROM(addr)</code>	string	Nil	Sender address
<code>ADDT0(addr)</code>	string	Nil	Add recipient (alias: SETTO)
<code>SETCC(addr)</code>	string	Nil	Add CC
<code>SETBCCT0(addr)</code>	string	Nil	Add BCC
<code>SET SUBJECT(subj)</code>	string	Nil	Subject line
<code>SETBODY(body)</code>	string	Nil	Email body
<code>SETATTACHMENT(path)</code>	string	Nil	Attachment file
<code>SEND()</code>	—	Bool	Send via net/smtp stdlib

4.14 MCPServer (Model Context Protocol)

Arquivo: `pkg/vm/mcp_native.go`

Função Nativa	Args	Retorno	Descrição
<code>MCPSERVER()</code>	—	Object	Create MCP server class
<code>NEW(name, [version])</code>	2 strings	self	Constructor
<code>ADDTOOL(toolName, description, schemaJSON, funcName)</code>	4 args	Nil	Register AdvPL function as MCP tool
<code>SERVE()</code>	—	Nil	Start stdio JSON-RPC server

4.15 WSRestServer (REST API)

Arquivo: `pkg/vm/rest_native.go`

Função Nativa	Args	Retorno	Descrição
<code>WSRESTSERVER()</code>	—	Object	Create REST server class
<code>NEW(name, [version])</code>	2 strings	self	Constructor (auto-registers annotated routes)
<code>ADDRROUTE(method, path, funcName)</code>	3 strings	Nil	Manual route registration
<code>GET()</code>	—	—	Auto-detected @Get routes
<code>POST()</code>	—	—	Auto-detected @Post routes
<code>PUT()</code>	—	—	Auto-detected @Put routes
<code>PATCH()</code>	—	—	Auto-detected @Patch routes

<code>DELETE()</code>	—	—	Auto-detected <code>@Delete</code> routes
<code>SERVE([addr/port])</code>	optional	Nil	HTTP listen (default :8080)
<code>SHUTDOWN()</code>	—	Nil	Graceful shutdown (5s timeout)

Annotation support: `@Get("/path")`, `@Post("/path")`, etc. scan bytecode at startup.

5. Framework MVC

Arquivos: `pkg/mvc/model.go` + `view.go` + `browse.go` + `types.go`

FWFormModel (`model.go`)

```
type FWFormModel struct {
    Name      string
    Fields    []*FieldDef
    Validations map[string][]ValidationRule
    PrimaryKey []string
}
```

Método	Args	Retorno	Descrição
<code>NewFWFormModel(name)</code>	string	*FWFormModel	Constructor
<code>AddField(field)</code>	*FieldDef	—	Adiciona campo
<code>AddValidation(fieldName, rule)</code>	2 strings	—	Validation rule
<code>Validate(data)</code>	map	error	Validates data against rules
<code>GetField(name)</code>	string	*FieldDef	Campo por nome
<code>SetPrimaryKey(keys...)</code>	...string	—	PK definition
<code>AddRequiredValidation()</code>	—	—	Required rule
<code>AddLengthValidation(min, max)</code>	2 ints	—	Length rule
<code>AddRangeValidation(min, max)</code>	2 values	—	Range rule
<code>AddCustomValidation(handler)</code>	func	—	Custom validator
<code>GetValidations()</code>	—	[]ValidationRule	All rules

FWFormView (`view.go`)

```
type FWFormView struct {
    Name      string
    Model     *FWFormModel
    Title     string
    Width     int
    Height    int
    Components []*Component
    Dialogs   []*Dialog
    MenuBar   *MenuBar
    ToolBar   *ToolBar
    StatusBar *StatusBar
    Events    map[string]EventHandler
}
```

Método	Args	Retorno	Descrição
<code>NewFWFormView(name, model)</code>	2 args	*FWFormView	Constructor
<code>AddComponent(comp)</code>	*Component	—	Add UI component
<code>AddDialog(dialog)</code>	*Dialog	—	Add dialog
<code>SetTitle()</code> / <code>SetSize()</code>	—	—	Title/size
<code>AddEvent(handler)</code>	*EventHandler	—	Event handler
<code>GetComponent(name)</code>	string	*Component	Get component
<code>SetValue()</code> / <code>GetValue()</code>	fieldName, val	—	Field get/set
<code>Validate()</code>	—	error	View validation
<code>TriggerEvent(eventName, context)</code>	2 args	error	Fire event
<code>AddOnChange/OnClick/OnFocus/OnBlur()</code>	—	—	Shortcut event adds

Component Types

`TButton` , `TGet` , `TComboBox` , `TCheckBox` , `TLabel` , `TPanel` , `TGroupBox` , `TTabs` , `TSplitter` , `TTreeView` , `TListView` , `TImage` , `TWindow` , `TSButton`

Dialog

```
type Dialog struct {
    Name, Title string
    Width, Height int
    When, Valid interface{}
    Components []*Component
    DefaultButton string
}
```

FWFormBrowse ([browse.go](#))

```
type FWFormBrowse struct {
    Name, Alias, Order, Filter, Title string
    Model *FWFormModel
    Columns []*BrowseColumn
    Fields []*BrowseField
    Events map[string]EventHandler
    Permissions struct { AllowAdd, AllowEdit, AllowDelete bool }
    Size, ReadOnly bool
    LineNumbers, MarkColumn bool
}
```

Método	Args	Retorno	Descrição
<code>NewFWFormBrowse(name, model)</code>	2 args	*FWFormBrowse	Constructor
<code>AddColumn(col)</code>	*BrowseColumn	—	Add column
<code>AddField(field)</code>	*BrowseField	—	Add field
<code>SetAlias/SetOrder/SetFilter/SetTitle/SetSize</code>	string	—	Configuration
<code>SetPermissions(add, edit, delete)</code>	3 bools	—	CRUD permissions
<code>AddEvent(handler)</code>	*EventHandler	—	Event handler

<code>Validate()</code>	—	error	Validation
<code>TriggerEvent(eventName, context)</code>	2 args	error	Fire event
<code>AddOnLineChange/OnDbClick/OnHeaderClick()</code>	—	—	Shortcut events

6. UI Desktop (Fyne)

Arquivos: `pkg/ui/` (11 arquivos)

FyneUIProvider (`provider.go`)

```
type FyneUIProvider struct {
    Window    fyne.Window
    Console    *OutputConsole
    FileTree   *FileTree
    Editor     *CodeEditor
    TreeView   *fyne.Container
    BrowseUI   vm.BrowseUI
    DialogUI   vm.DialogUI
}
```

Método	Args	Retorno	Descrição
<code>NewFyneUIProvider(window)</code>	fyne.Window	*FyneUIProvider	Constructor
<code>MsgInfo/MsgStop/MsgAlert/MsgYesNo</code>	variadic	—	Blocking dialogs
<code>Menu(items, title)</code>	[]string, string	int	Menu selection
<code>InputText(prompt, def, password)</code>	3 args	string	Text input dialog
<code>Browse(specJSON)</code>	[]byte	[]byte	FWMBrowse rendering
<code>Dialog(specJSON)</code>	[]byte	[]byte	MSDIALOG rendering

Renderer (`renderer.go`)

```
type Renderer struct {
    Window fyne.Window
}
```

Método	Args	Retorno	Descrição
<code>NewRenderer(window)</code>	fyne.Window	*Renderer	Constructor
<code>RenderFormView(view)</code>	*FWFormView	fyne.CanvasObject	Full form → Fyne container
<code>ShowFormView(view)</code>	*FWFormView	—	Create window + show
<code>RenderComponent(comp)</code>	*Component	fyne.CanvasObject	Single component → widget
<code>RenderButton(comp)</code>	*Component	*fyne.Button	Button widget
<code>RenderGet(comp)</code>	*Component	*entry.Entry	Text input widget
<code>RenderComboBox(comp)</code>	*Component	*widget.Box	ComboBox widget
<code>RenderCheckBox(comp)</code>	*Component	*widget.Check	Checkbox widget

<code>RenderLabel(comp)</code>	<code>*Component</code>	<code>*widget.Label</code>	Label widget
<code>RenderMenuBar(menuBar)</code>	<code>*MenuBar</code>	<code>*fyne.MainMenu</code>	Menu bar
<code>RenderMenu(item)</code>	<code>*MenuItem</code>	<code>*fyne.Menu</code>	Menu item/submenu
<code>RenderToolBar(toolBar)</code>	<code>*ToolBar</code>	<code>fyne.CanvasObject</code>	Toolbar
<code>RenderStatusBar(statusBar)</code>	<code>*StatusBar</code>	<code>fyne.CanvasObject</code>	Status bar

OutputConsole (`console.go`)

```
type OutputConsole struct {
    Label *widget.Label
    Scroll *container.Scroll
    Output []string
}
```

Método	Args	Retorno	Descrição
<code>NewOutputConsole()</code>	—	<code>*OutputConsole</code>	Constructor
<code>GetWidget()</code>	—	<code>fyne.CanvasObject</code>	Fyne widget
<code>Append(text)</code>	string	—	Add line
<code>Clear()</code>	—	—	Clear all

ConsoleWriter (`consolewriter.go`)

Método	Args	Retorno	Descrição
<code>NewConsoleWriter(console)</code>	<code>*OutputConsole</code>	<code>*ConsoleWriter</code>	Adapter constructor
<code>Write(p []byte)</code>	<code>[]byte</code>	(int, error)	io.Writer impl → Console append

FileTree (`filetree.go`)

```
type FileTree struct {
    List *widget.List
    Files []string
    Current string
    CurrentDir string
    OnSelect func(string)
}
```

Método	Args	Retorno	Descrição
<code>NewFileTree()</code>	—	<code>*FileTree</code>	Constructor
<code>GetWidget()</code>	—	<code>fyne.CanvasObject</code>	Fyne list widget
<code>SetCurrent(path)</code>	string	—	Set current dir
<code>GetCurrent()</code>	—	string	Current path
<code>SetFiles(files)</code>	<code>[]string</code>	—	Refresh listing
<code>SetOnSelect(callback)</code>	func(string)	—	Selection callback

<code>Refresh()</code>	—	—	Reload current dir
<code>GetSelectedFile()</code>	—	string	Selected path

CodeEditor (`editor.go`)

```
type CodeEditor struct {
    Entry    *highlightEntry
    Preview  *widget.RichText
    Overlay  *tapOverlay
    Stack    *fyne.Container
    Filename string
    Modified bool
}
```

Método	Args	Retorno	Descrição
<code>NewCodeEditor()</code>	—	*CodeEditor	Constructor
<code>GetContent()</code>	—	string	Source text
<code>SetContent(text)</code>	string	—	Replace source
<code>GetWidget()</code>	—	fyne.CanvasObject	Fyne widget
<code>SetFilename(name)</code>	string	—	Filename
<code>GetFilename()</code>	—	string	Current filename
<code>IsModified()</code>	—	bool	Dirty flag
<code>SetModified(modified)</code>	bool	—	Set dirty flag

Theme (`theme.go`)

Função	Args	Retorno	Descrição
<code>NewTheme()</code>	—	fyne.Theme	Custom AdvPP teal theme (#0f6e68)

7. UI Web (PO-UI)

Arquivo: `pkg/webui/server.go`

Server

```
type Server struct {
    SourceName string
    Run         RunFunc
    Sessions    map[string]*session
}
```

Função	Args	Retorno	Descrição
<code>New(sourceName, run)</code>	string, RunFunc	*Server	Constructor
<code>Serve(addr)</code>	string	error	HTTP server: static files + /events (SSE) + /reply (POST)
<code>Broadcast(kind, text)</code>	2 strings	—	SSE event to all sessions

Session (per-browser)

Método	Args	Retorno	Descrição
<code>ask(kind, msg, title)</code>	3 strings	string	Blocks until reply from browser
<code>askData(eventType, data)</code>	string, json.RawMessage	string	Structured data dialog
<code>reply(id, result)</code>	2 strings	—	Unblocks waiting ask

Provider (implements vm.UIProvider)

Método	Args	Retorno	Descrição
<code>MsgInfo/MsgStop/MsgAlert/MsgYesNo</code>	variadic	—	Dialogs sent via SSE
<code>Menu(items, title)</code>	[]string, string	int	Menu via SSE
<code>InputText(prompt, def, password)</code>	3 args	string	Input via SSE
<code>Browse(specJSON)</code>	[]byte	[]byte	FWMBrowse spec → action
<code>Dialog(specJSON)</code>	[]byte	[]byte	MSDIALOG spec → action

OutWriter

Método	Args	Retorno	Descrição
<code>Write(b []byte)</code>	[]byte	(int, error)	io.Writer → SSE "output" events

8. Banco de Dados SQLite

Arquivo: `pkg/db/sqlite.go`

Motor de bancos de dados SQLite com emulation de workarea Protheus.

Função/Método	Args	Retorno	Descrição
<code>NewSQLiteEngine(dbPath)</code>	string	(*SQLiteEngine, error)	Abre SQLite via shared helper
<code>SelectArea(alias)</code>	string	error	PRAGMA table_info + carrega TODOS records em memória
<code>Seek(key)</code>	string	(bool, error)	Linear search no primeiro campo
<code>Skip(count)</code>	int	error	Move ponteiro N registros
<code>GoTop()</code>	—	error	Posiciona no primeiro
<code>GoBottom()</code>	—	error	Posiciona no último
<code>EOF()</code>	—	bool	Fim do arquivo?
<code>BOF()</code>	—	bool	Antes do início?
<code>FieldGet(field)</code>	string	(Value, error)	Leitura de campo (AdvPL value)
<code>FieldPut(field, val)</code>	string, Value	error	Escrita de campo (only in-memory)
<code>RecLock()</code>	—	error	Placeholder (no-op)

<code>MsUnlock()</code>	—	error	COMMIT: UPDATE row by R_C_N_O_
<code>Append()</code>	—	error	INSERT blank row with type defaults + R_C_N_O_
<code>FieldPos(field)</code>	string	int	Posição do campo (1-based)
<code>RecCount()</code>	—	int	Total registros
<code>RecNo()</code>	—	int	Registro atual (1-based)
<code>QueryRows(query, args...)</code>	string, variadic	([]map[string]string, error)	Arbitrary SQL query
<code>Exec(query, args...)</code>	string, variadic	error	Execute SQL command
<code>Close()</code>	—	error	Close DB connection

Helpers internos

Função	Descrição
<code>convertDBValue(interface{}) Value</code>	SQL row → AdvPL value
<code>valueToSQL(Value) any</code>	AdvPL value → SQL parameter

9. Protocolos (DAP, MCP, REST)

DAP — Debug Adapter Protocol (`pkg/dap/`)

Função	Args	Retorno	Descrição
<code>NewConn(r, w)</code>	io.Reader, io.Writer	*Conn	Stdio connection with Content-Length framing
<code>ReadMessage()</code>	—	(*Envelope, error)	Block until complete DAP message
<code>SendResponse(reqSeq, cmd, success, msg, body)</code>	variadic	error	DAP response
<code>SendEvent(event, body)</code>	string, any	error	DAP event emission
<code>NewServer(conn, compile, attach)</code>	3 args	*Server	Launch mode DAP
<code>NewAttachServer(conn, sourcePath)</code>	2 args	*Server	Attach mode DAP
<code>OfferSession(v)</code>	*vm.VM	(bool, func(error))	Offer VM for debugging
<code>Run()</code>	—	error	Event loop

MCP — Model Context Protocol (`pkg/mcp/`)

Função	Args	Retorno	Descrição
<code>NewServer(name, version)</code>	2 strings	*Server	Constructor
<code>AddTool(t)</code>	Tool	—	Register/replace tool

<code>Serve(r, w)</code>	<code>io.Reader, io.Writer</code>	<code>error</code>	JSON-RPC stdio loop
--------------------------	-----------------------------------	--------------------	---------------------

Tool:

Campo	Tipo	Descrição
<code>Name</code>	string	Tool identifier
<code>Description</code>	string	Human-readable
<code>InputSchema</code>	map[string]any	JSON Schema
<code>Handler</code>	func(args) (string, error)	Implementation

Métodos JSON-RPC: `initialize`, `notifications/initialized`, `ping`, `tools/list`, `tools/call`

REST (`pkg/rest/`)

Função	Args	Retorno	Descrição
<code>NewServer(name, version)</code>	2 strings	*Server	Constructor
<code>AddRoute(r)</code>	Route	—	Register route
<code>Routes()</code>	—	[]Route	Introspection (sorted)
<code>Serve(addr)</code>	string	error	Blocking HTTP server
<code>Shutdown(ctx)</code>	context.Context	error	Graceful shutdown

Routing: `{param}` placeholders, query string + JSON body merge, GET `/_routes` introspection.

10. Build Standalone

Arquivo: `pkg/compiler/standalone.go`

Função	Args	Retorno	Descrição
<code>BuildStandaloneGUI(bc, outputFile, title, gui, buildLog)</code>	*Bytecode, 2 strings, bool, io.Writer	error	Embed bytecode in Go stub, compile → native executable. <code>gui</code> vem da flag <code>--gui</code> (ver Opções Globais acima) — baked in no stub via <code>__ADVPP_BUILT_AS_GUI__</code> e usado tanto para decidir o caminho de execução quanto para adicionar <code>-H=windowsgui</code> ao <code>go build</code> no Windows (<code>goBuildArgs</code>).

Stub Template Embutido

O template Go (`pkg/compiler/stub_template.go`) é embutido no build via string replace de placeholders (`__ADVPP_APP_TITLE__` , `__ADVPP_BUILT_AS_GUI__`) e compilado de verdade (`go build`) contra um `go.mod` gerado que faz `replace` do módulo AdvPP apontando pro checkout (`ADVPP_SRC` ou raiz do repo). Em runtime, o binário resultante escolhe um de dois caminhos:

1. Unmarshal `bytecode.json` embutido → `*Bytecode`
2. Varre o bytecode em busca de UI natives/classes/anotações REST (`hasUI`)
3. Decide o caminho:
 - **Console** se `!hasUI` , ou stdin for TTY (e o binário não foi `--gui / ADVPP_FORCE_GUI`), ou `ADVPP_HEADLESS_STANDALONE` setado —

anexa `TerminalUIProvider` se for TTY de fato, roda `v.Run()` diretamente e sai (sem Fyne, sem `app.New()`)

- **Fyne** nos demais casos: cria app Fyne com tema custom AdvPP, window com título especificado, console output widget, `VM(bc, traceMode=true)` com console writer + `FyneUIProvider` , DB factory (SQLite via `shared.ResolveStandaloneDatabasePath`), `v.Run()` em goroutine, `ShowAndRun()` bloqueia main; ao completar, `a.Quit()` e exit code

O caminho console nunca instancia `app.New()` /Fyne — importante para apps batch/CI que não podem depender de um display disponível.

11. Tabela de Opcodes do Bytecode

#	Opcode	Arg1	Arg2	Str	Descrição
0	OP_NIL	constIdx	—	—	Push nil constant
1	OP_TRUE	—	—	—	Push true
2	OP_FALSE	—	—	—	Push false
3	OP_NUMBER	constIdx	—	—	Push number constant
4	OP_STRING	constIdx	—	—	Push string constant
5	OP_DATE	constIdx	—	—	Push date constant
6	OP_LOAD_LOCAL	slotIdx	—	—	Load local var onto stack
7	OP_STORE_LOCAL	slotIdx	—	—	Store stack top into local
8	OP_LOAD_GLOBAL	nameIdx	—	constStr	Load global/dynamic var
9	OP_STORE_GLOBAL	nameIdx	—	constStr	Store global var
10	OP_LOAD_SELF	—	—	—	Push self reference (object methods)
11	OP_STORE_SELF	propIdx	—	propName	Store to self property
12	OP_NEW_ARRAY	size	—	—	Create array
13	OP_ARRAY_GET	—	—	—	array[idx] → stack
14	OP_ARRAY_SET	—	—	—	array[idx] := stack_top
15	OP_ARRAY_LEN	—	—	—	#array → stack
16	OP_NEW_OBJECT	pairCount	—	—	Create JSON-like object
17	OP_GET_PROP	propIdx	—	propName	object.prop → stack
18	OP_SET_PROP	propIdx	—	propName	stack → object.prop
19	OP_CALL_METHOD	argCount	—	methodName	obj:method(args)
20	OP_NEW_INSTANCE	classIdx	—	className	ClassName():New(args)
21	OP_CALL_FUNC	funcIdx	argCount	funcName	User function call
22	OP_CALL_NATIVE	nameIdx	argCount	funcName	Native function call
23	OP_RETURN	—	—	—	Return void

24	OP_RETURN_VALUE	—	—	—	Return stack top
25	OP_POP	—	—	—	Pop stack top
26	OP_JUMP	target	—	—	Unconditional jump
27	OP_JUMP_IF_FALSE	target	—	—	Conditional jump
28	OP_JUMP_IF_TRUE	target	—	—	Jump if true
29	OP_ADD	—	—	—	Addition / concat
30	OP_SUB	—	—	—	Subtraction
31	OP_MUL	—	—	—	Multiplication
32	OP_DIV	—	—	—	Division
33	OP_MOD	—	—	—	Modulo
34	OP_POW	—	—	—	Power
35	OP_NEG	—	—	—	Negation
36	OP_EQ	—	—	—	Equality
37	OP_NEQ	—	—	—	Not equal
38	OP_LT	—	—	—	Less than
39	OP_GT	—	—	—	Greater than
40	OP_LTE	—	—	—	Less/equal
41	OP_GTE	—	—	—	Greater/equal
42	OP_AND	—	—	—	Logical AND
43	OP_OR	—	—	—	Logical OR
44	OP_NOT	—	—	—	Logical NOT
45	OP_DOLLAR	—	—	—	\$ operator (contains)
46	OP_CONCAT	—	—	—	++ string concat
47	OP_NEW_CODEBLOCK	offset	—	—	Create code block closure
48	OP_EVAL_CODEBLOCK	—	—	—	Evaluate code block
49	OP_TRY_BEGIN	—	—	—	Begin try block
50	OP_TRY_END	—	—	—	End try block
51	OP_THROW	—	—	—	Throw exception
52	OP_CATCH	catchSlot	—	—	Catch variable slot
53	OP_DB_SELECT	aliasIdx	—	aliasName	Select work area
54	OP_DB_SEEK	—	—	—	Seek in current area
55	OP_DB_SKIP	count	—	—	Skip n records
56	OP_DB_GOTOP	—	—	—	Go to first

57	OP_DB_GOBOTTOM	—	—	—	Go to last
58	OP_EOF	—	—	—	Is EOF?
59	OP_BOF	—	—	—	Is BOF?
60	OP_FIELD_GET	fieldIdx	—	fieldName	Get field value
61	OP_FIELD_PUT	fieldIdx	—	fieldName	Set field value
62	OP_REC_LOCK	—	—	—	Lock record
63	OP_MS_UNLOCK	—	—	—	Unlock record
64	OP_MVC_NEW_MODEL	modelIdx	—	modelName	Create MVC model
65	OP_MVC_NEW_VIEW	viewIdx	—	viewName	Create MVC view
66	OP_MVC_NEW_BROWSE	browseIdx	—	browseName	Create MVC browse
67	OP_MVC_ADD_FIELD	fieldIdx	—	fieldName	Add field to model
68	OP_MVC_ADD_COMPONENT	compIdx	—	compName	Add component to view
69	OP_MVC_ADD_COLUMN	colIdx	—	colName	Add column to browse
70	OP_MVC_SET_PROPERTY	propIdx	—	propName	Set MVC property
71	OP_MVC_GET_PROPERTY	propIdx	—	propName	Get MVC property
72	OP_MVC_VALIDATE	—	—	—	Validate model
73	OP_MVC_SHOW	—	—	—	Show MVC screen
74	OP_MACRO	—	—	—	Macro substitution
75	OP_HALT	—	—	—	Terminate program
76	OP_DUP	—	—	—	Duplicate stack top
77	OP_SWAP	—	—	—	Swap top two stack items
78	OP_JUMP_IF_FALSE_OR_POP	target	—	—	Short-circuit pop
79	OP_POP_AND_JUMP	target	—	—	Conditional pop-jump
80	OP_NAMED_ARG	argNameIdx	—	paramName	Named arg marker
81	OP_FORLOOP_CMP	—	—	—	For-loop condition (step-aware)
82	OP_LOAD_UPVAL	upvalIdx	—	—	Load upvalue
83	OP_STORE_UPVAL	upvalIdx	—	—	Store upvalue
84	OP_LOAD_DYN	dynIdx	—	—	Load dynamic var
85	OP_STORE_DYN	dynIdx	—	—	Store dynamic var
86	OP_DECL_DYN	dynIdx	—	name	Declare dynamic (Private/Public)

12. Resumo de Coverage

O que REALMENTE Funciona (testado e validado)

Componente	Status	Evidência
Lexer	✔ Full	80+ keywords, CP-1252, bracket strings, dot literals, begin content
Parser	✔ Full	200+ AST nodes, Pratt precedence, DSL desugaring, named params
Codegen	✔ Full	70+ opcodes, closures, named args, IIF inline, constant dedup
VM Core	✔ Full	Stack machine, try/catch, dynamic scoping, operator overloading
Native Fns (strings)	✔ Full	35+ functions tested
Native Fns (math)	✔ Full	25+ functions tested
Native Fns (date/time)	✔ Full	14+ functions tested
Native Fns (arrays)	✔ Full	12+ functions tested including ASort/AEval/AScan with blocks
Native Fns (system)	✔ Full	Hash, file, env, sleep, findfunction, startjob
Native Fns (dialog)	✔ Full	MsgInfo/Stop/Alert/YesNo, Menu, InputText
Native Fns (file IO)	✔ Full	FCreate, FOpen, FReadStr, FWrite, FSeek, FClose, FError, Memoread/Memowrite
Native Fns (DB workarea)	✔ Full	SelectArea, Seek, Skip, GoTop, RecLock, MsUnlock, Append
Native Fns (raw SQL)	✔ Full	TCsqlExec, TCsqlQuery via SQLEngine interface
Native Fns (JSON)	✔ Full	JsonObject, GetNames, FromJson (parser real, <code>encoding/json</code>)
Native Fns (TUI/terminal)	✔ Full	UiBox, UiStreamBox/Reset, UiMarkdown (glamour), UiAltScreenEnter/Exit, UiTermWidth, ConOutRaw, ProcRun
Native Fns (MVC)	✔ Full	FwFormModel, FwFormView, FwFormBrowse, FwMBrowse
Native Fns (FWGridProcess)	✔ Full	Pool com threads, meters, afterExecute
Native Fns (geometry)	✔ Full	Vec add/sub/dot/cross/norm/normalize/dist/angle/scale/rotate
Native Fns (math stat)	✔ Full	Ceil, sinh/cosh/tanh, gcd/lcm/fact, mean/var/stddev/median, linreg, interp
Native Fns (LLM)	✔ Full	GGUF I2_S load, tokenize, generate, close
Native Fns (TMailMessage)	✔ Full	Setup SMTP + send
Native Fns (MCPServer)	✔ Full	AddTool + Serve sobre stdio
Native Fns (WSRestServer)	✔ Full	Routes + Serve com annotation scanning
SQLite Engine	✔ Full	Workarea emulation, field get/put, append, unlock (msunlock commit)
MsDialog renderer	✔ Full (web + desktop)	SAY/GET/BUTTON/BOX layout grid heuristic

FWMBrowse renderer	✔ Full (web + desktop)	Auto-CRUD from SX3 + PRAGMA
Web UI (SSE)	✔ Full	Per-session VM, async UI blocking, hot reload
Desktop UI (Fyne)	✔ Full	Editor, console, file tree, MVC renderer, dialogs
Tensor ops	✔ Full	2D math, matmul, transpose, reshape, reductions, linalg
Autograd	✔ Full	Tape-based reverse diff, SGD, Adam
Neural Network layers	✔ Full	Linear, Embedding with Forward/Params
DAP Debugger	✔ Full	Breakpoints, step, variables, stack frames
Macro substitution	✔ Full	&var and &(expr) pipeline
StartJob	✔ Full	Isolated VM per job, sync/async
CP-1252 conversion	✔ Full	Full 256-byte table lookup
Preprocessor	✔ Full	#include, #define, #ifdef, #xcommand, BEGINSQL
Bytecode serialization	✔ Full	Save/Load .abf JSON
Standalone build	✔ Full	Embed bytecode → native executable; console ou Fyne GUI (auto-detect, ou fixado via <code>--gui</code>)
Check (multi-file parallel)	✔ Full	Worker pool per CPU
Watch / Hot Reload	✔ Full	mtime polling 500ms

Stubs / Não Implementados (retornem valores placeholder)

Função	Retorno Stub	Observação
<code>GETMV(key, default)</code>	<code>default</code>	Parâmetros MV não configurados
<code>MAKEDIR(path)</code>	Nil	Não implementado
<code>PROCNAME()</code>	<code>" "</code>	Não implementado
<code>PROCLINE()</code>	<code>0</code>	Não implementado
<code>SETDATE(fmt)</code>	Nil	Apenas referencia
<code>SETCENT(n)</code>	Nil	Apenas referencia
<code>EMPNAME(cEmp)</code>	<code>"USER"</code>	Hardcoded stub
<code>FWALIASINDIC()</code>	<code>.F.</code>	Hardcoded stub
<code>FWMODEACCESS()</code>	<code>1</code>	Hardcoded stub
<code>FWHASACCMODE()</code>	<code>.T.</code>	Hardcoded stub
<code>FWLIBVERSION()</code>	<code>"1.0.0"</code>	Hardcoded version
Todas funções <code>MP*</code>	Nil/Bool/String stub	Módulos MultiPrint não carregados

<code>FWRESTAREA/FWGETAREA/etc.</code>	Nil/String/Array	Framework apps stubs
<code>ChangeQuery/TcSqlExec/TcSqlQuery</code>	Nil/Bool	Via SQLEngine interface em browse.go, não no workarea DBEngine
Variadas funções FW* de framework Protheus	Valores placeholder	Funções de UI complexa (FWTreeView, FWTabs, etc.)
<code>FWTreeview</code>	Nil (stub)	Não mapeado na VM (classe não registrada)

Observações Importantes

1. **CP-1252 obrigatório:** Todos os fontes `.prw` / `.prg` / `.tlpp` DEVEM estar em Windows-1252. O conversor automático está incluído no pipeline, mas depende da tabela `cp1252ToUTF8`.
2. **D_E_L_E_T_ = '':** Todas as queries de leitura filtram soft-delete implicitamente (via `DBCLOSEAREA` e browse engine).
3. **Filial:** `xFilial('XXX')` retorna `""` (stub) — projetos reais devem configurar o valor no código.
4. **Thread safety:** VMs de diferentes jobs são totalmente isoladas (nenhum estado compartilhado). O mesmo banco SQLite é aberto por múltiplas conexões (WAL mode).
5. **Operator overloading:** Objetos podem definir métodos `OPERATOR_ADD`, `OPERATOR_SUB`, etc. que são invocados automaticamente.
6. **Code blocks closures:** Suporte completo a upvalues — code blocks acessam variáveis do escopo pai via `OP_LOAD_UPVAL` / `OP_STORE_UPVAL`.
7. **Named arguments:** Chamadas suportam `func(paramNome := valor)` — a VM reordena via `reorderNamedArgs()`.
8. **IIF/IF inline:** Compiladas com short-circuit — só avalia o ramo escolhido (não é uma função biasa).